Contents

1. Giriş ... 3
   1.1. Neden Buradayız? (Motivasyon) ... 3
   1.2. Yol Haritası ve Süreç ... 4
   1.3. Beklentiler ve Metodoloji ... 4
   1.4. Başlangıç Vuruşu ... 4
   1.5. Node.js Kurulumu ... 5
   1.6. TypeScript Nedir? (Superset Kavramı) ... 5
   1.7. Tarayıcı Sorunsalı ve Derleme (Compile) ... 5
   1.8. TypeScript Kurulumu (Global Install) ... 6
   1.9. Çalışma Ortamını Hazırlama ... 6
   1.10. İlk Kod ve Tip Tanımlama ... 7
   1.11. Manuel Derleme (Transpilation) ... 7
   1.12. Çıktıyı Analiz Etme (TS vs JS) ... 8
   1.13. Oluşan JavaScript'i Çalıştırma ... 8
   1.14. Target Ayarı (ECMAScript Versiyonları) ... 9
   1.15. İzleme Modunun Çalışma Mantığı ... 11
   1.16. Canlı Test (Live Demo) ... 11
   1.17. İkinci Terminal İhtiyacı ... 11
   1.18. Kurtarıcı: `ts-node` ... 12
   1.19. `ts-node` Kurulumu ve İlk Test ... 12
   1.20. Temizlik ve İlk Değişken ... 13
   1.21. Hata Yapma Lüksü (Kırmızı Çizgi) ... 14
   1.22. Explicit Typing (Açık Tip Tanımlama) ... 14
   1.23. Kaçış Kapısı: `any` Tipi ... 15
   1.24. Dizilere (Arrays) Giriş ... 15
   1.25. Generic Yazım Tarzı (`Array<Type>`) ... 16
   1.26. Tuples (Demetler) ... 16
   1.27. Tuple Nerede Kullanılır? ... 17
      1.27.1. Objeler (Objects) ve Tip Çıkarımı ... 17
      1.27.2. Inline Type Definition (Satır İçi Tanımlama) ve Zorluklar ... 18
      1.27.3. Problem Tanımı: 'Sihirli Sayılar' (Magic Numbers) ... 18
      1.27.4. Enum (Numaralandırma) Nedir? ... 19
      1.27.5. Fonksiyonların Disiplini ... 19
      1.27.6. Enum Tanımlama ve Kullanma ... 20
      1.27.7. Arka Planda Neler Oluyor? (Numeric Enums) ... 20
      1.27.8. String Enums (Metin Bazlı Enumlar) ... 21
      1.27.9. Fonksiyon Parametrelerini Tipleme ... 21
      1.27.10. Dönüş Tipleri (Return Types) ... 22
      1.27.11. Hiçlik Tipi: 'void' ... 22
      1.27.12. Opsiyonel Parametreler (?) ... 23
      1.27.13. Varsayılan Parametreler (Default Parameters) ... 24
      1.27.14. Modern Yazım: Arrow Functions (Ok Fonksiyonları) ... 24
      1.27.15. Rest Parameters (Toplayıcı Parametreler) ... 25
      1.27.16. Fonksiyon Tipleri (Function Types) ... 25
      1.27.17. Callback Fonksiyonları Tipleme ... 26

1.27.18. Birleşim Tipleri (Union Types) . . . 27
1.27.19. Tip Daraltma (Type Narrowing) . . . 27
1.27.20. Literal Types (Tam Değer Tipleri) . . . 28
1.27.21. Type Aliases (Tip Takma Adları) . . . 28
1.27.22. Objeler için Type Alias . . . 29
1.27.23. Type vs Interface (Kısa Giriş) . . . 30
1.27.24. Interface Tanımlama . . . 30
1.27.25. Kalıtım (Extends) ve Kod Tekrarını Önleme . . . 31
1.27.26. Declaration Merging (Beyan Birleştirme) . . . 31
1.27.27. Type vs Interface: Hangisini Seçelim? . . . 32
1.27.28. Sınıflara (Classes) Giriş . . . 32
1.27.29. Constructor (Kurucu Metot) . . . 33
1.27.30. Metot Ekleme (Davranış) . . . 33
1.27.31. Sınıfı Kullanma (Instance Oluşturma) . . . 34
1.27.32. Erişim Belirleyiciler (Access Modifiers) . . . 35
1.27.33. Private'ın Mantığı . . . 36
1.27.34. Constructor Kısayolu (NestJS İçin Çok Kritik!) . . . 36
1.28. 3. Ders . . . 37
1.28.1. Generics (Jenerikler). . . . 37
1.28.2. Problem: 'Tekrarlayan Kod' veya 'Any Tuzağı' . . . 37
1.28.3. `any` Kullanırsak Ne Olur? . . . 38
1.28.4. Çözüm: `<T>` ile Generic Tanımlama . . . 38
1.29. 3. Ders . . . 39
1.29.1. Generic Interface Kullanımı (API Response Örneği) . . . 39
1.29.2. Generic Classes (Veri Deposu Örneği) . . . 40
1.29.3. Constraints (Kısıtlamalar - `extends`) . . . 41
1.29.4. Decorators (Dekoratörler) Giriş . . . 41
1.29.5. Decorator Nedir? (Teknik Tanım) . . . 42
1.29.6. Ayar: `experimentalDecorators` . . . 42
1.29.7. İlk Class Decorator'ımızı Yazıyoruz . . . 42
1.29.8. Decorator Factories (Parametre Alan Dekoratörler) . . . 43
1.29.9. Method Decorators (Metot Dekoratörleri) . . . 44
1.29.10. Metodu Manipüle Etmek (Descriptor.value) . . . 44
1.29.11. Property Decorators (Özellik Dekoratörleri) . . . 45
1.29.12. Metadata Mantığı (Etiketleme) . . . 46
1.29.13. Parameter Decorators (Parametre Dekoratörleri) . . . 46
1.29.14. Spagetti Koddan Kurtuluş: Modülerlik . . . 47
1.29.15. Export ve Import Temelleri . . . 47
1.29.16. Named Exports vs Default Exports . . . 48
1.29.17. Barrel Dosyaları (`index.ts`) . . . 48
1.29.18. Senkron vs Asenkron (Beklemek ya da Beklememek) . . . 49
1.29.19. Promise (Söz Verme) Yapısı . . . 49
1.29.20. Modern Çözüm: Async / Await . . . 50
1.29.21. NestJS İçin Neden Kritik? . . . 51
1.29.22. Asenkron Hata Yönetimi (Try-Catch) . . . 51
1.29.23. Async Fonksiyonların Dönüş Tipi (Promise Wrapper) . . . 52
1.29.24. Promise.all (Paralel İşlemler) . . . 52

1.30. Profesyonel Geliştirme Ortamı ... 53
1.30.1. Giriş ve “Kırık Camlar Teorisi” ... 53
1.30.2. Prettier Nedir? Neden Lazım? ... 53
1.30.3. Prettier Kurulumu ... 54
1.30.4. Konfigürasyon Dosyası (`.prettierrc`) ... 54
1.30.5. VS Code Entegrasyonu (Format On Save) ... 55
1.30.6. İkinci Ajan: ESLint (Kod Polisi) ... 55
1.30.7. Prettier vs ESLint Farkı ... 55
1.30.8. ESLint Kurulumu ... 56
1.30.9. ESLint Yapılandırma Dosyası (.eslintrc) ... 56
1.30.10. Kuralları Test Etmek ... 57
1.30.11. Polisin Girmemesi Gereken Yerler (.eslintignore) ... 57
1.30.12. Büyük Çatışma: Prettier vs ESLint ... 57
1.30.13. Klasör Yapısını Düzenleme (`src` ve `dist`) ... 58
1.30.14. TSConfig Ayarları (`rootDir` ve `outDir`) ... 58
1.30.15. Otomasyon: Nodemon Kurulumu ... 59
1.30.16. Nodemon Ayarları (nodemon.json) ... 59
1.30.17. NPM Scripts (Otomasyon Komutları) ... 60
1.30.18. Geliştirme Ortamı Komutları (`start:dev`) ... 60
1.30.19. Canlı Ortam Komutları (`build & start:prod`) ... 61
1.30.20. Linting (Sağlık Kontrolü) ... 61
1.30.21. Git Başlatma (Versiyon Kontrolü) ... 61
1.30.22. .gitignore Dosyası Oluşturma ... 62
1.30.23. İlk Kayıt (Commit) ... 62
1.31. 1. Hafta Proje Ödevi Tanıtımı ... 62
1.31.0.1. Teknik Gereksinimler ... 63
1.31.0.2. Kod Kalitesi ve Kısıtlamalar ... 63
1.31.0.3. Teslim Formatı ... 63
1.31.0.4. Enum ve Interface Tanımlama ... 63

1. Giriş

Herkese merhabalar, hepiniz hoş geldiniz arkadaşlar!

Enerjiler nasıl? Umarım herkes yerindedir. Bugün, kariyerinizde teknik anlamda çok ciddi bir seviye atlamanızı sağlayacak, uzun soluklu ve bir o kadar da keyifli bir yolculuğun ilk adımını atıyoruz.

Önümüzdeki 56 ders saati boyunca, modern backend dünyasının şu an en popüler, en çok aranan ve en güçlü teknoloji yığınlarından (stack) birini; yani TypeScript ve NestJS ikilisini A'dan Z'ye, tüm detaylarıyla öğreneceğiz.

1.1. Neden Buradayız? (Motivasyon)

Peki, neden buradayız? Neden sadece JavaScript veya sadece Node.js değil de, **TypeScript** ve **NestJS**?

Şöyle düşünün: Bir inşaat yapacaksınız. JavaScript ile harika bir kulübe yapabilirsiniz, çok hızlı yaparsınız. Ama 50 katlı bir gökdelen dikmek istediğinizde, o kulübede kullandığınız yöntemler yetersiz kalır, bina sallanmaya başlar. İşte TypeScript, o binanın çelik iskeletidir. Size hata yapma lüksünü azalttırır, kodunuzu daha güvenli hale getirir.

NestJS ise o binanın mimari planıdır. Nereye mutfak koyacaksınız, elektrik tesisatı nereden geçecek, asansör boşluğu nerede olacak... NestJS bize bu düzeni verir. Biz bu kursta, ***"çalışsın da nasıl çalışırsa çalışsın"*** kodları değil, kurumsal firmaların, büyük projelerin talep ettiği o ***"mimari"*** yapıyı kurmayı öğreneceğiz.

1.2. Yol Haritası ve Süreç

Kısaca yol haritamızdan bahsedeyim, neyle karşılaşacaksınız bilin.

İlk 3 haftayı tamamen TypeScript'e ayıracağız. Eğer aranızda "Ben JavaScript biliyorum, TypeScript'e gerek var mı?" diyenler varsa, onlara 3. haftanın sonunda fikrini tekrar soracağım. O **"loose" (gevşek)** yapıdan, **"strongly typed" (sıkı tipli)** yapıya geçtiğimizde, editörün size nasıl yardım ettiğini gördüğünüzde bir daha asla düz JS yazmak istemeyeceksiniz.

Sonrasında ise NestJS maceramız başlayacak. Controller'lar, Servisler, Modüller... Hepsini konuşacağız. Veritabanı tarafında ise MySQL kullanacağız. Docker gibi karmaşık DevOps süreçlerine girmeden, doğrudan bilgisayarımızda çalışan, gerçek bir ilişkisel veritabanı (RDBMS) üzerinde verilerimizi yöneteceğiz. SQL sorgularını, tablolar arası ilişkileri NestJS üzerinden, TypeORM kullanarak nasıl yönetiriz, hepsini göreceğiz.

1.3. Beklentiler ve Metodoloji

Bu eğitimde benim sizden beklentim şu: Lütfen hata yapmaktan korkmayın. Burası bizim laboratuvarımız. ***"Hocam burada saçma bir soru sorar mıyım?"*** diye düşünmeyin, en kötü soru sorulmamış sorudur.

Derslerimiz 45 dakika sürecek, blok ders yapmayacağız ki zihinlerimiz taze kalsın. Bol bol kod yazacağız. Slaytlar üzerinden okuyup geçmek yok. Ben burada editörümü açıp kod yazarken, sizden de aynısını yapmanızı, hatayı benimle beraber almanızı ve o hatayı beraber çözmemizi istiyorum. Çünkü yazılım, izleyerek değil, o hatayı debug ederek öğrenilir.

1.4. Başlangıç Vuruşu

Eğer herkes hazırsa, kahvelerinizden, sularınızdan birer yudum alın. Arka planda açık kalan gereksiz sekmeleri kapatalım, odağımızı toplayalım.

İlk işimiz, silahlarımızı kuşanmak. Bilgisayarlarımızda Node.js kurulu mu? VS Code'umuz hazır mı? Hemen bunları kontrol ederek ve TypeScript'in ne olduğunu, neden **"JavaScript'in süper seti"** dendiğini konuşarak başlıyoruz.

Hadi bakalım, editörler açılsın, ilk dersimiz başlasın!

1.5. Node.js Kurulumu

Evet, şimdi ilk teknik kontrolümüzle başlıyoruz. Lütfen herkes Visual Studio Code'unu veya kullandığı terminali açsın.

Biz TypeScript yazacağız evet, ama TypeScript'i bilgisayarımıza kurmak ve yönetmek için Node.js ve onun paket yöneticisi olan **NPM**'e ihtiyacımız var. O yüzden ilk iş, Node.js kurulu mu değil mi, bir bakalım.

Terminalinize şu komutu yazmanızı istiyorum: `node -v` . Enter'a bastığınızda karşınıza v18, v20, v22 gibi bir versiyon numarası çıkması lazım.

Eğer versiyon numarası görüyorsanız harika, gemideyiz demektir. Eğer ***"command not found"*** veya ***"böyle bir komut yok"*** hatası alıyorsanız, hemen nodejs.org adresine gidip LTS (Long Term Support) sürümünü indirip kurmanız gerekiyor. Bu dersin devamı için bu şart.

1.6. TypeScript Nedir? (Superset Kavramı)

Kurulumlar kontrol edilirken, size TypeScript'in ne olduğunu kafanızda somutlaştırmak istiyorum. Ezber tanım şu: **"TypeScript, JavaScript'in supersetidir."**

Peki ne demek bu **Superset**?

Hayal edin arkadaşlar... Bir tane daire çizin kafanızda. Bu dairenin adı JavaScript. Bu dairenin içinde ne var? Değişkenler, fonksiyonlar, döngüler... Bildiğiniz her şey.

Şimdi bu dairenin etrafına, onu tamamen kapsayan daha büyük bir daire çizin. İşte bu dıştaki büyük daire TypeScript.

Bu ne anlama geliyor? Şu anlama geliyor: Yazdığınız her geçerli JavaScript kodu, aynı zamanda geçerli bir TypeScript kodudur. Yani TypeScript, JavaScript'i dışlamaz; onu kapsar, kucaklar ve üzerine ekstra özellikler ekler. O yüzden TypeScript öğrenirken aslında JavaScript bilginizi çöpe atmıyorsunuz, ona ***"süper güçler"*** ekliyorsunuz.

1.7. Tarayıcı Sorunsalı ve Derleme (Compile)

Ama burada çok kritik bir ayrım var. Burayı iyi dinleyin, çünkü geliştirme yaparken sık sık kafanız karışmasın.

Ben buraya harika bir TypeScript kodu yazdım diyelim. İçinde tipler var, interface'ler var... Bunu kopyaladım, Google Chrome'un konsoluna yapıştırdım. Çalışır mı?

Hayır, çalışmaz. Patlar.

Neden? Çünkü tarayıcılar (Chrome, Safari, Firefox) TypeScript bilmezler. Onların ana dili JavaScript'tir.

İşte bu yüzden biz kodumuzu TypeScript ile yazacağız, sonra araya bir ***"tercüman"*** koyacağız. Bu tercüman bizim TypeScript kodumuzu alacak, tarayıcının veya Node.js'in anlayacağı saf JavaScript'e çevirecek. Biz bu işleme Compilation (Derleme) veya daha teknik tabiriyle **Transpilation** diyoruz.

Yani biz TS yazacağız, bilgisayar bunu JS'e çevirecek, ve çalışan şey aslında JS olacak. Bizim güvenli limanımız TS, ama motorun yakıtı JS.

1.8. TypeScript Kurulumu (Global Install)

Mantığı anladığımıza göre, hadi şu tercümanı, yani TypeScript derleyicisini bilgisayarımıza kuralım.

Terminaller tekrar önümüzde. Node.js yüklü demiştik. Şimdi şu sihirli komutu yazıyoruz. Bu komut, TypeScript'i bilgisayarınızın her yerinden erişilebilir hale getirecek.

Komutumuz şu : `npm install -g typescript`

Buradaki `-g` , ***"global"*** demek. Yani ***"Bu paketi sadece bu klasöre değil, bilgisayarımın ana kütüphanesine kur ki, yarın başka bir projede de direkt kullanabileyim"*** diyoruz.

`Enter` 'a basın ve kurulumun bitmesini bekleyin. İnternet hızınıza göre 5-10 saniye sürecektir.

Kurulum bittikten sonra, gerçekten kuruldu mu diye teyit etmemiz lazım. Hemen sağlamasını yapalım. Şu komutu yazın:

`tsc -v`

`tsc` , TypeScript Compiler'ın kısaltması. Eğer karşınıza `Version 5.x.x` gibi bir şey çıktıysa, tebrikler! Artık bilgisayarınız TypeScript dilini konuşabiliyor.

Harika. Kurulumu tamamladık, `tsc` komutunun çalıştığını teyit ettik. Şimdi teoriyi bırakıp, parmaklarımızı koda değdirme vakti. İlk dosyamızı oluşturup o ***sihirli*** dönüşümü gözlerimizle göreceğiz.

1.9. Çalışma Ortamını Hazırlama

Evet arkadaşlar, madem `tsc` yani derleyicimiz hazır, hadi ilk laboratuvar ortamımızı kuralım.

Sizden ricam şu: Masaüstünüze veya projelerinizi tuttuğunuz klasöre gidin. İsmi `TS-Giris` olan boş bir klasör oluşturun. Türkçe karakter kullanmamaya, boşluk bırakmamaya (tire kullanın) özen gösterin; eliniz alışsın.

Şimdi bu klasöre sağ tıklayıp **"Open with Code"** diyerek VS Code ile açalım. VS Code kullanmayanlar da kendi editörlerinden bu klasörü açsınlar.

Ekranınızda bomboş bir proje var şu an. Sol taraftaki dosya gezginine (Explorer) gelin. İlk dosyamızı oluşturuyoruz. Ama dikkat! Uzantımız artık `.js` değil.

Dosyanın adını a `app.ts` koyalım. Bakın `.ts` yazar yazmaz VS Code'un ikon seti hemen değişti, o mavi TS logosunu gördünüz. Artık TypeScript dünyasındayız.

1.10. İlk Kod ve Tip Tanımlama

Şimdi `app.ts` dosyasının içine girelim.

Az önce ne demiştim? ***"Her JavaScript kodu geçerli bir TypeScript kodudur."*** Hadi bunu test edelim. Bildiğimiz düz JavaScript yazalım:

```
console.log("Merhaba NestJS Sınıfı!");
```

Gördüğünüz gibi, editör kızmadı, altını çizmedi. Çünkü bu geçerli. Ama biz buraya TypeScript öğrenmeye geldik, değil mi? Hadi TypeScript'in o meşhur **Tip Güvenliği (Type Safety)** özelliğini kullanalım.

Şu kodu yazmanızı istiyorum:

```
let mesaj: string = "Bu benim ilk TS kodum";
```

Bakın burada ne yaptık? Değişken isminden (`mesaj`) hemen sonra iki nokta üst üste koyduk ve `string` yazdık.

Bunun meali şudur: ***"Ey bilgisayar, bu mesaj kutusunun içine sadece ve sadece YAZI (string) koyabilirsin. Yarın öbür gün ben veya bir başkası buraya sayı koymaya kalkarsa, o eli kır... yani hata ver.***

Normal JavaScript'te bunu yapamazdınız. JS'te bir değişkene önce yazı, sonra sayı, sonra uçak, sonra kedi atayabilirsiniz; gıkını çıkarmaz. Ama TypeScript disiplinlidir.

1.11. Manuel Derleme (Transpilation)

Kodumuzu yazdık, kaydettik **(CTRL+S)**. Şimdi kritik soru şu: Bu kodu nasıl çalıştıracağız?

Node.js, `.ts` dosyasını doğrudan çalıştıramaz arkadaşlar. İnanmıyorsanız terminali açın, `node app.ts` yazın. Hata alacaksınız. Çünkü Node.js, o yazdığımız `: string` kısmını anlamaz. "***Bu ne syntax'ı?*** " der.

Ne yapmamız lazım? Tercümanı çağırmamız lazım. Yani bu `.ts` dosyasını `.js` dosyasına çevireceğiz.

VS Code'un üst menüsünden `Terminal → New Terminal` deyin. Ve şu komutu yazın:

```
tsc app.ts
```

`Enter` tuşuna basın.

Ekranda bir şey yazmadı, değil mi? Terminal bir çıktı vermedi. Linux felsefesinde "***No news is good news***" (Haber yoksa iyi haberdir) kuralı geçerlidir. Hata vermediyse çalışmıştır.

Ama sol taraftaki dosya gezginine (Explorer) dikkatlice bakmanızı istiyorum. Orada yeni bir arkadaş belirdi.

1.12. Çıktıyı Analiz Etme (TS vs JS)

Gördünüz mü? `app.js` dosyası kendiliğinden oluştu. Biz oluşturmadık, `tsc (derleyici)` oluşturdu.

Hadi şimdi ekranı ikiye bölelim. Bir tarafta `app.ts` açık kalsın, diğer tarafta `app.js` dosyasını açın. Aradaki farkı görebilen var mı?

Bakın `app.ts` dosyasında: `let mesaj: string = "..."` yazıyor.

Ama app.js dosyasına bakın: `var mesaj = "..."` (veya `let/const` ayarına göre değişir).

O `: string` ibaresi yok olmuş! Nereye gitti?

İşte TypeScript'in sırrı bu: TypeScript sadece geliştirme aşamasında (Development Time) vardır. Derleyici kodunuzu kontrol eder, tip hatası yapmadığınızdan emin olur, her şey güvenliyse o tip tanımlarını siler ve geriye saf, tarayıcının veya Node.js'in anlayacağı tertemiz bir JavaScript bırakır.

Yani canlıya (Production) çıktığımızda sunucuda çalışan kod TypeScript değildir, TypeScript'in ürettiği bu JavaScript'tir.

1.13. Oluşan JavaScript'i Çalıştırma

Şimdi elimizde ne var? Bir `app.ts` var, bir de onun çocuğu olan `app.js` var.

Bizim nihai amacımız bu kodu çalıştırmaktı. Hadi terminale geri dönelim. Az önce node `app.ts` yazıp hata almıştık hatırlarsanız. Şimdi şunu yazın : `node app.js` . `Enter` tuşuna basın.

İşte şimdi konsolda ***"Merhaba NestJS Sınıfı!"*** yazısını gördük. (Veya kodunuzda ne yazdıysanız).

Döngümüz tamamlandı arkadaşlar. Döngü şu:

1. Sen TypeScript yazarsın.
2. `tsc` komutuyla bunu derlersin.
3. Çıkan sonucu Node.js ile çalıştırırsın.

Burası çok temel ama çok kritik. Node.js sizin TypeScript dosyanızın yüzüne bile bakmaz, o sadece çıkan JS ile ilgilenir.

Tamam, döngüyü kurduk ama... İçinizden ***"Hocam bu iş böyle mi gidecek?"*** diyenler vardır eminim.

Düşünsenize, projemizde 50 tane dosya var. `user.ts` , `product.ts` , `order.ts` ... Her kod değiştirdiğimizde terminale gidip: `tsc user.ts` , `tsc product.ts` ... diye tek tek yazacak mıyız?

Bu tam bir angarya. Hatta işkence. Yazılım mühendisleri tembeldir arkadaşlar, yani akıllıca tembeldir. Sürekli tekrar eden işleri sevmezler.

Bizim öyle bir şey yapmamız lazım ki, sadece ***"Derle"*** diyelim ve o bütün projedeki TypeScript dosyalarını bulsun, ayarlarımıza göre hepsini tek seferde derlesin.

İşte bunun için, projemize bir **Yönetici** atamamız lazım. Bu yöneticinin adı: `tsconfig.json` .

Şimdi bu yönetici dosyasını elle oluşturmayacağız. TypeScript'in bize sunduğu bir sihirbaz var.

Terminalinizi temizleyin (`clear` veya `cls`). Şu komutu yazın arkadaşlar, bu komutu bu kurstan sonraki hayatınızda yüzlerce kez kullanacaksınız:

```shell
tsc --init
```

Listing 1 : index.php

`Enter` tuşuna basın.

Terminal size "***Successfully created a tsconfig.json file***" dedi. Hemen sol tarafa, dosya gezginine bakın. `app.ts` ve `app.js` dosyalarının yanına `tsconfig.json` geldi.

Bu dosyayı açın bakalım içinde ne varmış.

Korkutucu görünüyor değil mi? Yüzlerce satır var. Ama dikkatli bakarsanız çoğu satırın başında `//` var. Yani onlar yorum satırı, pasif durumdalar. Microsoft bize tüm ayarları listelemiş ama çoğunu kapalı getirmiş. Bize sadece en önemlilerini açık bırakmış.

1.14. Target Ayarı (ECMAScript Versiyonları)

Bu dosyadaki ayarlardan en önemlisine, yani `target` ayarına odaklanmanızı istiyorum. Genelde en üstlerde olur.

`"target": "es2016"` (veya versiyona göre es5/es6) gibi bir şey yazar.

Bu ne demek biliyor musunuz? Bu, tercümanın hedef dili.

Şöyle düşünün: Siz modern bir Türkçe ile bir mektup yazdınız (TypeScript). Tercümana diyorsunuz ki: ***"Bunu İngilizceye çevir."*** Ama hangi İngilizceye? Shakespeare döneminin

İngilizcesine mi (es5 - eski tarayıcılar için)? Yoksa günümüz Modern İngilizcesine mi (es2016 ve üzeri - modern Node.js için)?

Eğer hedef kitleniz Internet Explorer 11 gibi antika tarayıcılarsa, burayı es5 yaparsınız. Derleyici, modern kodlarınızı alır, eski tarayıcıların anlayacağı o çirkin, uzun kodlara dönüştürür.

Ama biz bu kursta Node.js tarafındayız, sunucu tarafındayız. Bizim sunucumuz modern. O yüzden burayı `es2016` , `es2020` hatta `esnext` tutabiliriz. Şimdilik varsayılan ayarda kalsın, dokunmayalım. Sadece mantığını anlayın yeter.

Şimdi terminale geri dönelim. Hatırlarsanız az önce `tsc app.ts` yazmıştık. Yani derleyiciye ***"Sadece şu dosyaya bak"*** demiştik.

Ama artık elimizde bir `tsconfig.json` var. Bu dosya projenin kök dizininde (root) duruyor. Bu ne demek? Artık derleyiciye tek tek dosya adı vermemize gerek yok demek.

Terminali temizleyin ve sadece şunu yazın:

```shell
tsc
```

Listing 2 : index.php

Yanına dosya adı yok. Uzantı yok. Sadece `tsc`. Ve `Enter` tuşuna basın.

Ne oldu? Yine ses yok, hata yok. Ama arka planda şu gerçekleşti: TypeScript derleyicisi çalıştı, klasördeki `tsconfig.json` dosyasını buldu, oradaki kuralları okudu ve projedeki **tüm** `.ts` dosyalarını (şu an sadece `app.ts` var ama 100 tane de olsa fark etmezdi) bulup hepsini tek seferde `.js` dosyasına çevirdi.

Artık projenizdeki 50 dosyayı tek komutla derleyebilirsiniz.

Tamam, tek komuta düşürdük ama hala bir sorunumuz var. Ben koda gidip bir virgül değiştirsem, tekrar terminale gelip `tsc` yazmak zorundayım. Sonra tekrar `node app.js` yazmak zorundayım.

Yazılım geliştirirken biz yüzlerce kez kaydet (CTRL+S) yaparız. Her seferinde terminale komut yazarsak gün biter. Bize, biz dosyayı kaydettiğimiz **an** devreye giren bir sistem lazım. İşte buna **Watch Mode (İzleme Modu)** diyoruz.

Terminale şu komutu yazmanızı istiyorum:

```shell
tsc -w
```

Listing 3 : index.php

Buradaki `-w` (tire w), **watch** yani **izle** demek. `Enter` tuşuna basın.

1.15. İzleme Modunun Çalışma Mantığı

`Enter` tuşuna bastığınızda fark ettiyseniz terminal bu sefer komutu bitirip yeni satıra geçmedi. İmleç yanıp sönüyor ve ekranda muhtemelen şöyle bir şey yazıyor:

```shell
Starting compilation in watch mode... Found 0 errors.
Watching for file changes.
```

Listing 4 : index.php

Şu an derleyici pusuya yattı arkadaşlar. Aynen bir güvenlik kamerası gibi `app.ts` dosyanızı izliyor. Siz dokunmadığınız sürece o da kıpırdamaz.

Ama dosyanızda en ufak bir değişiklik yapıp kaydettiğiniz anda, o bunu milisaniyeler içinde fark edip, arka planda `app.js` dosyasını güncelleyecek.

Bu terminal penceresi artık **meşgul**. Buraya başka komut yazamazsınız. Burası bizim **Derleme Terminalimiz** oldu.

1.16. Canlı Test (Live Demo)

Hadi test edelim. İnanmak için görmek lazım.

Terminal açık kalsın, `app.ts` dosyasına gidin. `let mesaj: string = ...` satırındaki yazıyı değiştirin. Mesela: ***"Otomatik derleme harika bir şey!"*** yazın.

Gözünüz aşağıda, terminalde olsun. Şimdi **CTRL+S (Kaydet)** yapın.

Gördünüz mü? Terminalde anlık bir hareketlenme oldu:

```shell
File change detected. Starting incremental compilation...
```

Listing 5 : index.php

dedi ve hemen bitti.

Şimdi gidin `app.js` dosyasını açın (veya açıksa bakın). İçindeki `var mesaj = ...` kısmının da otomatik olarak değiştiğini göreceksiniz.

İşte modern geliştirme deneyimi budur. Siz sadece TypeScript yazmaya odaklanırsınız, arka plandaki hamallar (derleyiciler) sizin için JavaScript üretir.

1.17. İkinci Terminal İhtiyacı

Derleyicimiz şu an pusuya yatmış durumda, her kaydettiğimizde `app.js` güncelleniyor. Buraya kadar süper.

Ama bir sorunumuz var arkadaşlar. Terminalimiz meşgul! `tsc -w` komutu terminali kilitledi. Peki biz bu kodu nasıl çalıştırıp sonucunu göreceğiz?

Çare : İkinci Terminal.

VS Code'da sağ alttaki artı (+) ikonuna basarak veya terminal menüsünden **New Terminal** diyerek ikinci bir pencere açın. Artık iki terminaliniz var. Birincisi: `tsc` (Derleme işçisi), ikincisi: Bizim komut alanımız.

İkinci terminalde `node app.js` yazıp Enter tuşuna basın. Çıktıyı gördünüz. Şimdi koda gidin, değiştirin, kaydedin. Birinci terminal işini yaptı. Tekrar ikinci terminale gelin, tekrar `node app.js` yazın.

Çalışıyor mu? Evet. Pratik mi? Kesinlikle hayır.

Fark ettiyseniz, kod otomatik derlense bile, Node.js otomatik olarak yeniden başlamıyor. Her değişiklikten sonra bizim gidip manuel olarak ***"Hadi Node, tekrar çalış"*** dememiz gerekiyor.

Yani döngümüz şu an şöyle: **Kod Yaz -> Kaydet -> (Otomatik Derleme) -> Terminale Tıkla -> Yukarı Ok Tuşu -> Enter**.

Günde 1000 kere kod değiştirdiğinizi düşünün. Günde 1000 kere o terminale tıklayıp Enter tuşuna basmak... Bu da bir angarya. Biz geliştiriciler olarak klavyeden elimizi kaldırmayı sevmeyiz.

Bize öyle bir araç lazım ki; biz kodu kaydettiğimiz anda hem derlesin, hem de kodu çalıştırsın. Aradaki `.js` dosyasıyla falan uğraşmayalım. Direkt TypeScript dosyasını çalıştırıyormuş gibi hissedelim.

1.18. Kurtarıcı: `ts-node`

İşte burada sahneye NestJS geliştirirken de en yakın dostumuz olacak bir araç çıkıyor: `ts-node`.

`ts-node` nedir? Adından anlayacağınız üzere, TypeScript (ts) ve Node.js (node) kelimelerinin birleşimi.

Bu araç şunu yapar: Sizin `.ts` dosyanızı alır, hafızada (memory) anında derler ve diskte fiziksel bir `.js` dosyası oluşturmadan direkt olarak çalıştırır.

Yani sanki Node.js, TypeScript biliyormuş gibi davranır. Bu araç **Production (Canlı)** ortamı için değildir, orası için yine build alacağız. Ama **Development (Geliştirme)** ortamında hızımıza hız katar.

1.19. `ts-node` Kurulumu ve İlk Test

Hadi şu aracı kuralım. Yine terminale ihtiyacımız var. O boşta duran (`tsc`'nin çalışmadığı) ikinci terminali kullanabilirsiniz. Veya `tsc` çalışan terminali **CTRL+C** ile durdurup orayı kullanabilirsiniz. Artık `tsc`'yi manuel çalıştırmayacağız çünkü.

Şu komutu yazın:

```shell
npm install -g ts-node
```

Listing 6 : index.php

Yine `-g` (global) dedik, çünkü bunu her projede kullanacağız.

Kurulum bitti mi? Süper. Şimdi sihir zamanı. `tsc` komutunu unutun. `node` komutunu unutun. Aradaki `app.js` dosyasını bile silebilirsiniz, gerek yok.

Sadece şunu yazın:

```shell
ts-node app.ts
```

Listing 7 : index.php

ve `Enter` tuşuna basın.

Bum! Çıktı direkt karşınızda. Ne derleme bekledik, ne `app.js` aradık. Sanki doğrudan TypeScript çalıştırdık. İşte bu, bizim geliştirme motorumuz olacak.

1.20. Temizlik ve İlk Değişken

Araçlarımız hazır olduğuna göre, artık kod yazabiliriz. Lütfen `app.ts` dosyasının içindekileri tamamen silin. Temiz bir sayfa açalım. Şimdi, TypeScript'in ne kadar zeki olduğunu test edeceğiz. Ekrana şu kodu yazmanızı istiyorum:

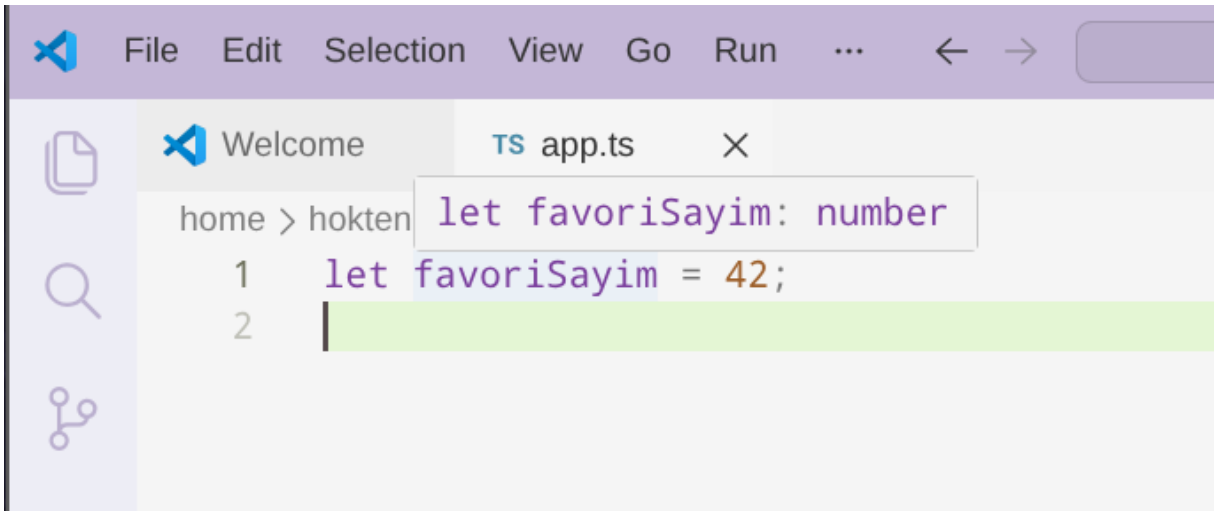
```ts
let favoriSayim = 42;
```

Listing 8 : index.php

Dikkat edin, hiçbir tip belirtmedim. `: number` falan yazmadım. Sadece standart JavaScript yazdım.

Şimdi sizden ricam şu: Farenizi `favoriSayim` kelimesinin üzerine getirin ve bekleyin.

VS Code size ne gösteriyor? `let favoriSayim: number` yazıyor, değil mi?



Ben ona number demedim, ama o `42`'nin bir sayı olduğunu gördü ve ***"Hah, demek ki bu kutunun tipi number olacak"*** dedi. İşte buna teknik dilde **Type Inference (Tip Çıkarımı)** diyoruz. TypeScript, bağlamdan tipi anlayacak kadar zekidir.

1.21. Hata Yapma Lüksü (Kırmızı Çizgi)

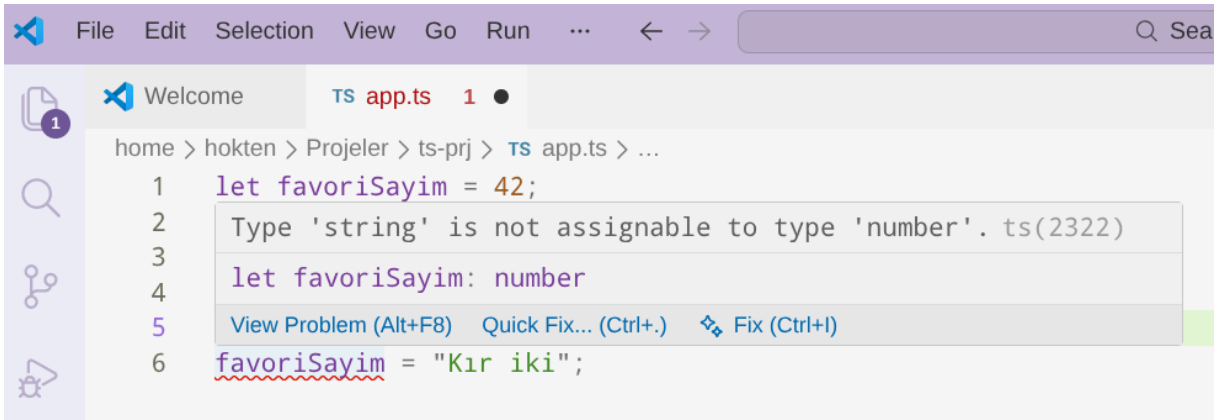
Peki bu çıkarım ne işimize yarar? Bizi korur. Hemen bir alt satıra geçin ve şunu yazın:

```ts
favoriSayim = "Kırk iki";
```

Listing 9 : index.php

Bunu yazar yazmaz, altının kırmızı tırtıklı bir çizgiyle çizildiğini göreceksiniz. Hani Word'de imla hatası yaparsınız ya, aynen öyle.

Üzerine gelin hatanın. Ne diyor? **Type 'string' is not assignable to type 'number'**. (String tipi, number tipine atanamaz.)



Bakın daha kodu çalıştırmadık. `ts-node` yazmadık. Kaydetmedik bile. Ama TypeScript anında müdahale etti: ***"Dur!"*** dedi. ***"Sen yukarıda buna sayı demiştin (veya ben öyle anlamıştım), şimdi yazı koymaya çalışıyorsun. Yapamazsın."***

İşte JavaScript'te olmayan, bizim bu kurs boyunca sırtımızı yaslayacağımız güvence budur arkadaşlar.

1.22. Explicit Typing (Açık Tip Tanımlama)

Bazı durumlarda işi TypeScript'in tahminine bırakmak istemeyiz. Tipi açık açık, yani **Explicit (Açık)** olarak belirtmek isteriz. Buna da **Type Annotation** denir.

Şimdi şu değişkenleri benimle birlikte tanımlayın. Bunlar bizim **"Kutsal Üçlü"**müzdür (Primitives):

1. **String (Metin) :** `let ad: string = "NestJS";`
2. **Number (Sayı) :** `let dersSaati: number = 56;` (Not: TypeScript'te `int` , `float` , `double` ayrımı yoktur. Hepsi `number` dır.)

3. **Boolean (Mantıksal) :** `let egitimBittiMi: boolean = false;`

Bunları yazdığınızda kodunuz daha okunabilir hale gelir. Başka bir geliştirici koda baktığında, o değişkenin ne işe yaradığını, içinde ne tutması gerektiğini şak diye anlar.

1.23. Kaçış Kapısı: `any` Tipi

Peki hocam, ben bir değişken tanımlayacağım ama içine ne geleceğini bilmiyorum? Belki veritabanından sayı gelecek, belki `null` gelecek, belki bir obje gelecek. Ne yapacağım?

Burada TypeScript bize bir ***"Kaçış Kapısı"*** sunar. Ama bu kapı tehlikelidir.

Şunu yazın: `let belirsizDeger: any = 5;` . Şimdi altına geçin: `belirsizDeger = "Merhaba";` . (Hata vermez) `belirsizDeger = true;` (Hata vermez). `belirsizDeger.olmayanBirFonksiyon();` (Yine hata vermez!)

`any` tipi, TypeScript'in ***"Tamam pes ediyorum, ne yaparsan yap, karışmıyorum"*** dediği tiptir. Yani o değişkende TypeScript özelliğini kapatır, bildiğiniz JavaScript'e döner.

Eğitim boyunca kuralımız şu: Çok zorda kalmadıkça `any` kullanmak yasak. Çünkü `any` kullanacaksak, neden TypeScript öğreniyoruz ki?

1.24. Dizilere (Arrays) Giriş

Değişkenleri öğrendik ama gerçek hayatta veriler genelde tek başına gezmez, sürüler halinde gezer. Ürün listesi, kullanıcı listesi, yorumlar...

JavaScript'te diziler **ortaya karışık** tabak gibidir. İçine sayı da atarsın, yazı da atarsın, kimse **ne yapıyorsun** demez. Ama TypeScript'te biz düzeni severiz.

Hadi bir dizi tanımlayalım. `app.ts` dosyanıza şunu yazın:

```ts
let hobiler: string[] = ["Yüzme", "Kodlama", "Müzik"];
```

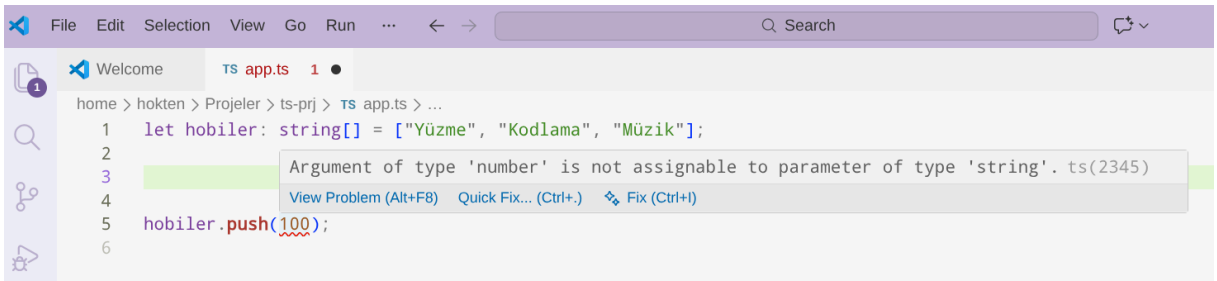
Listing 10 : index.php

Buradaki `string[]` (string ve köşeli parantez) ifadesi şu demek: ***"Bu bir dizidir ve içinde SADECE stringler (yazılar) olabilir."***

Hadi bunu bozmaya çalışalım. Alt satıra geçin ve şunu yazın:

```ts
hobiler.push(100);
```

Bam! Altı hemen kızardı. TypeScript diyor ki: ***"Hoppala... Sen bana bu kutunun içinde sadece yazılar olacak diye söz verdin, şimdi içine sayı (100) atmaya çalışıyorsun. Kabul etmem."***



1.25. Generic Yazım Tarzı (`Array<Type>`)

Dizileri tanımlamanın bir yolu daha var arkadaşlar. Bunu da göstermek zorundayım çünkü NestJS dokümantasyonunu okurken sık sık karşınıza çıkacak. Kafanız karışmasın.

Şunu yazalım:

```ts
let notlar: Array<number> = [85, 90, 100];
```

Listing 11

Bakın az önceki `number[]` yazımı ile `Array<number>` yazımı arasında teknik olarak hiçbir fark yoktur. İkisi de aynı kapıya çıkar.

İkinci yazıma **Generic (Jenerik)** yazım denir. Java veya C# geçmişi olanlar buna daha aşinadır. ***"Array türünden bir yapı, ama tipi number"*** şeklinde okunur. Biz genelde kısa olanı (`number[]`) kullanacağız ama diğerini görürseniz de şaşırmayın.

1.26. Tuples (Demetler)

Peki arkadaşlar, size bir soru...

Benim öyle bir dizim olsun ki, birinci elemanı mutlaka bir **ID (sayı)** olsun, ikinci elemanı ise mutlaka bir **İsim (string)** olsun. Ve bu düzen asla şaşmasın. Yani veritabanından gelen bir satır gibi.

JavaScript'te bunu garanti edemezsiniz. Ama TypeScript'te **Tuple (Demet)** dediğimiz özel bir yapı var.

Hadi deneyelim:

```ts
let rol: [number, string] = [1, "Admin"];
```

Listing 12

Dikkat edin, köşeli parantezin içine tipleri tek tek yazdım. Bu ne demek? Birinci eleman `number` olmak **ZORUNDA**. İkinci eleman `string` olmak **ZORUNDA**. Ve bu dizi sadece 2 elemanlı olabilir (başlangıç atamasında).

Eğer siz `[1, "Admin"]` yerine `["Admin", 1]` yazarsanız, yani sırayı karıştırırsanız, TypeScript hata verir. Çünkü sıra **(sequence)** önemlidir.

1.27. Tuple Nerede Kullanılır?

Hocam bu Tuple nerede işimize yarayacak?

Mesela bir fonksiyonunuz var, geriye iki değer döndürmek istiyor. Biri işlem sonucu (`success: boolean`), diğeri de mesaj (`message: string`). Bunu bir obje (`object`) olarak da döndürebilirsiniz ama Tuple olarak döndürmek bazen çok daha pratiktir.

Örneğin, React kullananlar `useState` hook'unu bilir. `const [count, setCount] = useState(0)` İşte oradaki yapı aslında bir Tuple mantığıdır. Birinci eleman değer, ikinci eleman fonksiyon. Sırası bellidir, değişmez.

NestJS tarafında da bazen CSV dosyalarını okurken veya Excel satırlarını işlerken ***"1. sütun ad, 2. sütun soyad"*** gibi kesin kurallarımız olduğunda Tuple kullanacağız.

1.27.1. Objeler (Objects) ve Tip Çıkarımı

Son olarak, JavaScript dünyasının en temel taşına, yani Objelere değinelim.

JavaScript'te obje oluşturmak çocuk oyuncağıdır. TypeScript'te de öyle, ama kuralları vardır.

Hemen `app.ts` dosyasına şu basit objeyi yazın:

```ts
const ogrenci = {
    ad: "Ahmet",
    yas: 25
};
```

Listing 13

Yine tip belirtmedik. Ama TypeScript hemen taradı ve anladı. Farenin imlecini `ogrenci` değişkeninin üzerine getirdiğinizde şunu göreceksiniz:

`{ ad: string; yas: number; }`

TypeScript bu objenin "***şeklini***" **(shape)** ezberledi.

Şimdi altına şunu yazmaya çalışın:

`ogrenci.yas = "yirmi beş";` -> **HATA!** (Sayı bekliyordu, yazı verdiniz).

Daha ilginci, şunu yazmaya çalışın:

`ogrenci.email = "ahmet\@mail.com";` -> **HATA!**

Neden? Çünkü TypeScript diyor ki: '***Sen bu objeyi ilk oluşturduğunda sadece*** `ad` ***ve*** `yas` ***özellikleri vardı.*** `email` ***diye bir şey yoktu. Kafana göre sonradan özellik ekleyemezsin.***'

JavaScript'te bunu yapardınız, ama TypeScript'te objenin şeması baştan bellidir.

1.27.2. Inline Type Definition (Satır İçi Tanımlama) ve Zorluklar

Peki hocam, ben bu objenin tipini elle yazmak istesem nasıl görünürdü?

Şöyle görünürdü (Bunu sadece izleyin, yazmanıza gerek yok, yorucu çünkü):

```ts
let kitap: { baslik: string; sayfaSayisi: number; yazar: string } = {
    baslik: "Suç ve Ceza",
    sayfaSayisi: 600,
    yazar: "Dostoyevski"
};
```

Listing 14

Bakın ne kadar çirkin ve uzun oldu değil mi? İki nokta üst üste koyduktan sonra süslü parantez açıp tek tek tipleri yazdık.

Eğer 50 tane özelliğimiz olsa, kodumuz okunmaz hale gelirdi. Üstelik bu yapıyı başka bir yerde tekrar kullanmak istesek, kopyala-yapıştır yapmamız gerekirdi.

İşte bu sorunu çözmek için önümüzdeki ders, TypeScript'in en güçlü silahlarından biri olan **Interface (Arayüz)** ve **Type Alias** konularına gireceğiz. Bu karmaşık yapıları tek bir isim altında toplayıp paketlemeyi öğreneceğiz.

İlk derste ne yaptık? TypeScript'in polis memuru gibi başımızda durduğunu, 'Buraya sayı giremezsin, buraya yazı yazamazsın' diyerek bizi disipline soktuğunu gördük. Dizi (Array) ve Demet (Tuple) yapılarına baktık.

Şimdi bu derste vitesi biraz artırıyoruz. Artık sadece "***veri tutmakla***" kalmayacağız, o verileri "***anlamlı hale getirmeyi***" ve verileri işleyen fabrikaları, yani "***fonksiyonları***" nasıl denetim altına alacağımızı öğreneceğiz.

1.27.3. Problem Tanımı: 'Sihirli Sayılar' (Magic Numbers)

Konuya girmeden önce size bir senaryo çizeyim.

Bir e-ticaret siteniz var. Siparişlerin durumunu veritabanında tutuyorsunuz.

0	Beklemede
1	Hazırlanıyor
2	Kargoda
3	Teslim Edildi

Kodun içinde şöyle bir if bloğu yazdığınızı düşünün: `if (siparis.durum === 2) { ... }`

Şimdi, bu kodu 6 ay sonra açıp okuduğunuzda veya ekibe yeni katılan bir arkadaş okuduğunda ne diyecek? "*Bu 2 ne demek? Hata mı? İptal mi? Kargo mu?*"

İşte yazılımda biz buna **Magic Number (Sihirli Sayı)** diyoruz ve bundan nefret ederiz. Kodun içinde ne olduğu belli olmayan sayılar dolaşmamalı. İşte TypeScript burada imdadımıza `ENUM` yapısıyla yetişiyor.

1.27.4. Enum (Numaralandırma) Nedir?

`Enum` , yani **Enumeration**, aslında sayıları veya metinleri isimlendirmemize yarayan bir yapıdır.

Biz kodumuzda `2` yazmak yerine `SiparisDurumu` .Kargoda yazacağız. Arka planda o yine `2` olacak, bilgisayar için fark etmeyecek. Ama biz insanlar için kod okumak şiir okumak gibi olacak.

Bu derste ilk işimiz bu Enum yapısını kurmak olacak. Sadece sayılarla değil, String Enum'larla da (Örn: 'TR', 'EN', 'DE') nasıl çalışacağımızı göreceğiz.

1.27.5. Fonksiyonların Disiplini

Dersin ikinci yarısında ise Fonksiyonlara el atacağız.

JavaScript'te fonksiyon yazarken parantezin içine `(x, y)` yazar geçeriz. Peki `x` ne? Sayı mı? Obje mi? Fonksiyon geriye ne döndürüyor? Bir sayı mı döndürüyor yoksa hiçbir şey döndürmüyor mu (`void`)?

TypeScript ile fonksiyonlarımızın giriş kapısına ve çıkış kapısına güvenlik görevlisi dikeceğiz. Yanlış veri giremeyecek, yanlış veri çıkamayacak.

Hazırsanız, VS Code'larımız açık kalsın. `app.ts` dosyamızı temizleyelim. İlk başlığımız: **Enums**.

Hadi şu " `Sihirli Sayılar` " sorununu, bir kargo takip sistemi simülasyonu ile kökünden çözelim. Başlıyoruz!

Önce "***Kötü***" Yolu Görelim

Hadi önce şu problemi kod üzerinde canlı canlı yaşayalım ki çözümün kıymetini anlayalım.

`app.ts` dosyanız boş, değil mi? Şunu yazın arkadaşlar:

```ts
// KÖTÜ YÖNTEM
let siparisDurumu = 2;

if (siparisDurumu === 2) {
    console.log("Siparişiniz kargoya verildi.");
}
```

Listing 15

Şimdi bu koda bakın. Sizce bu kod çalışır mı? Evet, bal gibi çalışır. `ts-node app.ts` derseniz ekranda yazıyı görürsünüz.

Ama sorun şu: Ben bu projeye 3 ay sonra geri döndüğümde, `2` ne demekti? Kargoda mıydı? İptal miydi? Yoksa `"Müşteri iade etti"` mi demekti? Hatırlama şansım sıfır. Dokümantasyona gidip bakmam lazım. İşte bu "*Magic Number*" sorunudur. Kodun okunabilirliğini öldürür.

1.27.6. Enum Tanımlama ve Kullanma

Şimdi, TypeScript'in bize sunduğu Enum yapısı ile bu kaosu düzene sokalım.

Hemen kodun en üstüne gelin ve şu yapıyı tanımlayın. Dikkat edin, `enum` anahtar kelimesiyle başlıyoruz ve genelde isimlendirmede **Baş Harfi Büyük (PascalCase)** kullanırız.

```ts
enum SiparisDurumu {
    Beklemede,
    Hazirlaniyor,
    Kargoda,
    TeslimEdildi
}
```

Listing 16

Harika. Şimdi az önceki değişkenimizi güncelleyelim. O çirkin `2` sayısını siliyoruz.

```ts
let benimSiparisim: SiparisDurumu = SiparisDurumu.Kargoda;

if (benimSiparisim === SiparisDurumu.Kargoda) {
    console.log("Sipariş yola çıktı!");
}
```

Listing 17

Farkı hissettiniz mi? Artık `if` bloğunu okurken "*Eğer sipariş durumu Kargoda ise...*" diye cümle kurabiliyoruz. Kod kendini anlatıyor.

1.27.7. Arka Planda Neler Oluyor? (Numeric Enums)

Peki arkadaşlar, size bir soru.

Şu an `benimSiparisim` değişkeninin içinde gerçekte ne var? "*Kargoda*" yazısı mı var?

Hadi görelim. Kodun en altına şunu ekleyin ve çalıştırın:

`console.log(benimSiparisim);`

Terminalde ne gördünüz? `2` gördünüz, değil mi?

Neden? Çünkü Enum'lar, aksi belirtilmedikçe `0` 'dan başlayarak otomatik numaralandırır.

Beklemede = 0

Hazırlanıyor = 1

Kargoda = 2

TeslimEdildi = 3

TypeScript bunu arka planda sayı olarak tutar ki performanslı olsun. Ama bize yazarken kelime olarak gösterir ki okunabilir olsun.

1.27.8. String Enums (Metin Bazlı Enumlar)

Hocam iyi güzel de, ben veritabanına `'2'` kaydetmek istemiyorum. Yarın öbür gün araya "***Paketlendi***" diye bir durum eklesem bütün sayılar kayacak, veritabanı çöp olacak!

Haklısınız. İşte bu yüzden biz genelde `String Enum` kullanırız. Yani sayı değil, metin atarız.

Enum tanımımızı şöyle değiştirelim:

```ts
enum SiparisDurumu {
    Beklemede = "BEKLEMEDE",
    Hazirlaniyor = "HAZIRLANIYOR",
    Kargoda = "KARGODA",
    TeslimEdildi = "TESLIM_EDILDI"
}
```

Listing 18

Bunu yaptığınızda, artık `console.log` çıktınız `2` değil, `KARGODA` olacaktır.

NestJS projelerimizde de genelde bu `String Enum` yapısını kullanacağız çünkü veritabanında (MySQL'de) okunabilir veri tutmak hayat kurtarır.

1.27.9. Fonksiyon Parametrelerini Tipleme

Enumlar cepte. Şimdi temiz bir sayfa açalım veya kodun altına devam edelim. Konumuz: **Fonksiyonlar**.

JavaScript'te fonksiyonlar biraz " `Giriş Serbest` " tabelası asılmış bir eğlence merkezi gibidir. İçeri kimin girdiği belli değildir.

Şu koda bakın :

```js
function topla(a, b) {
    return a + b;
}
```

Listing 19

Ben bu fonksiyonu `topla(5, 10)` diye çağırırsam `15` döner. Süper. Ama `topla(5, 'Elma')` diye çağırırsam ne olur? JavaScript "*Hayır*" demez. `5Elma` sonucunu verir. Matematiksel bir işlem yapıyorsam bu bir felakettir.

Hadi TypeScript ile fonksiyonun kapısına güvenlik dikelim. Yazın bakalım:

```ts
function topla(a: number, b: number) {
    return a + b;
}
```

Listing 20

Şimdi bu fonksiyonu çağırmayı deneyin: `topla(5, "Elma");`

Anında kırmızı çizgi! TypeScript diyor ki: "*Hop kardeşim!* `b` *parametresi sayı olmak zorunda, sen metin veriyorsun. Geçemezsin.*" İşte **Parametre Tiplemesi** budur.

1.27.10. Dönüş Tipleri (Return Types)

Giriş kapısını tuttuk. Peki ya çıkış kapısı?

Fonksiyonun işlemi bitince dışarıya ne fırlattığı da çok önemlidir. Biz buna **Return Type** diyoruz.

Genelde TypeScript, `return a + b` dediğinizde sonucun `number` olacağını anlar (Inference). Ama biz ne demiştik? "*Explicit*" **(Açık)** olmak her zaman daha iyidir.

Fonksiyonumuzu şöyle güncelleyelim:

```ts
function topla(a: number, b: number): number {
    return a + b;
}
```

Listing 21

Parantezden hemen sonra gelen `: number` , bu fonksiyonun dışarıya mutlaka bir sayı fırlatacağının taahhüdüdür.

Eğer süslü parantezlerin içinde return `"Sonuç 15"` derseniz, TypeScript kızar. " `Sen sayı döneceğim diye söz verdin, neden yazı dönüyorsun?` " der. Bu özellik, dalgınlıkla yanlış veri döndürmenizi engeller.

1.27.11. Hiçlik Tipi: 'void'

Peki hocam, her fonksiyon bir şey döndürmek zorunda mı? Mesela sadece ekrana log basan veya veritabanına kayıt atıp işlemi bitiren bir fonksiyonumuz varsa?

Bunun cevabı: `void` .

Void, 'boşluk', 'hiçlik' veya 'geçersiz' demektir. Eğer bir fonksiyon `return` anahtar kelimesini kullanmıyorsa, onun dönüş tipi voiddir.

Hemen deneyelim:

```ts
function hataGoster(mesaj: string): void {
    console.log("HATA: " + mesaj);
    // return yok!
}
```

Listing 22

Eğer bu fonksiyonun içine `return 5;` yazmaya çalışırsanız hata alırsınız. Çünkü void, "*Buradan dışarı veri çıkışı yok*" demektir. NestJS'te Controller metodlarında bunu sık sık kullanmayacağız (çünkü orada veri döneriz) ama arkaplan işlerinde (background jobs) çok işimize yarayacak.

1.27.12. Opsiyonel Parametreler (?)

Son olarak, fonksiyonlarda çok sevdiğimiz bir özellik: **Opsiyonel (Seçimli) Parametreler**.

Bazen bir parametre gelse de olur, gelmese de olur. Mesela kullanıcı adını ve soyadını alan bir fonksiyonumuz var. Soyad zorunlu olmasın istiyoruz.

Bunu yapmak için parametrenin sonuna soru işareti (`?`) koyarız.

```ts
function selamla(ad: string, soyad?: string): void {
    if (soyad) {
        console.log("Merhaba " + ad + " " + soyad);
    } else {
        console.log("Merhaba " + ad);
    }
}
```

Listing 23

Dikkat edin: `soyad` parametresinin yanındaki soru işareti, "*Bu parametre* `string` *olabilir* ***VEYA*** `undefined` *olabilir*" demektir.

Artık bu fonksiyonu iki şekilde de çağırabilirim:

1. `selamla("Ahmet", "Yılmaz");` -> Çalışır.
2. `selamla("Ahmet");` -> Çalışır.

Tek bir kural var: Opsiyonel parametreler her zaman **EN SONDA** olmalıdır. Zorunlu parametrelerin arasına opsiyonel koyamazsınız.

1.27.13. Varsayılan Parametreler (Default Parameters)

Opsiyonel parametrelerde (`?`) bir sorunumuz var: Eğer kullanıcı değeri göndermezse, o değişkenin değeri `undefined` (tanımsız) oluyor. Ve biz kodun içinde sürekli `if (soyad === undefined)` diye kontrol yapmak zorunda kalıyoruz.

Bunun yerine, "*Eğer kullanıcı bir değer göndermezse, şu değeri kullan*" diyebilmek çok daha temizdir. Buna **Default Parameter** diyoruz.

Hemen kodlayalım:

```ts
function puanVer(oyuncuAdi: string, puan: number = 100): void {
    console.log(oyuncuAdi + " oyuncusunun puani: " + puan);
}
```

Listing 24

Bakın puan: `number = 100` dedik. Soru işareti yok!

Şimdi deneyelim:

`puanVer("Mehmet", 50);` -> Ekrana 'Mehmet oyuncusunun puani: 50' yazar. (Bizim verdiğimiz değeri kullandı).

`puanVer("Ayşe");` -> Ekrana 'Ayşe oyuncusunun puani: 100' yazar. (Değer vermedik, varsayılanı kullandı).

Bu yöntem `undefined` kontrolü yapmaktan çok daha güvenli ve okunaklıdır. NestJS'te özellikle konfigürasyon ayarlarında bunu çok sık kullanacağız.

1.27.14. Modern Yazım: Arrow Functions (Ok Fonksiyonları)

"Şimdiye kadar hep klasik function anahtar kelimesini kullandık. Ama modern dünyada, özellikle React veya NestJS tarafında kod yazarken daha kısa, daha şık bir yöntem olan Arrow Function'ları tercih ediyoruz.

Mantık aynı, sadece yazım şekli (syntax) farklı.

Hadi az önceki toplama fonksiyonunu 'Ok' formatına çevirelim:

```ts
const carp = (a: number, b: number): number => {
    return a * b;
};
```

Listing 25

Gördünüz mü? `function` kelimesi gitti, yerine `=>` (eşittir ve büyüktür) işareti geldi.

Hatta eğer fonksiyonunuz tek satırsa, süslü parantezleri ve return kelimesini de atabilirsiniz. Şuna bakın:

`const kisaCarp = (a: number, b: number): number ⇒ a * b;`

Ne kadar temiz değil mi? Bu yazım tarzına elinizi alıştırın arkadaşlar. Çünkü NestJS'te bir diziyi filtrelerken veya haritalarken (map/filter) sürekli bu okları kullanacağız.

1.27.15. Rest Parameters (Toplayıcı Parametreler)

Fonksiyonlarla ilgili son ve çok güçlü bir özellikten bahsedeceğim.

Diyelim ki bir toplama fonksiyonu yazacaksınız. Ama kullanıcı 2 sayı da gönderebilir, 5 sayı da, 100 sayı da... `topla(a, b, c, d, e ... )` diye sonsuza kadar parametre mi yazacağız? Hayır.

Burada devreye **Rest Parameters** giriyor. Yani "*Geriye kalan her şeyi bir dizi (array) olarak topla*" diyoruz.

Söz dizimi şu: Üç nokta `...`

```ts
function hepsiniTopla(...sayilar: number[]): number {
    let toplam = 0;
    // Gelen bütün sayıları döngüyle topluyoruz
    sayilar.forEach((sayi) => {
        toplam += sayi;
    });
    return toplam;
}
```

Listing 26

Parametreye dikkat: `... sayilar: number[]` . TypeScript'e diyoruz ki: "*Bana kaç tane parametre gelirse gelsin, hepsini al, sayilar adında bir diziye paketle.*"

Kullanımı: `hepsiniTopla(10, 20`); -> Sonuç 30. `hepsiniTopla(1, 2, 3, 4, 5);` -> Sonuç 15.

Bu yapı, esnek fonksiyonlar yazmak için harikadır.

1.27.16. Fonksiyon Tipleri (Function Types)

Şimdi arkadaşlar, biraz beyin yakalım ama çok önemli bir konuya değinelim.

JavaScript'te (ve dolayısıyla TypeScript'te) fonksiyonlar da aslında birer objedir, birer değerdir. Yani bir değişkene sayı atayabildiğiniz gibi, bir fonksiyonu da atayabilirsiniz.

Peki, bir değişkenin tipini "*Bu değişken sadece şu yapıda bir fonksiyon tutabilir*" diye nasıl kısıtlarız?

Hadi şu örneği yazalım:

```ts
let islem: (s1: number, s2: number) => number;
```

Listing 27

Bu satır ne anlama geliyor?

1. `islem` adında bir kutum (değişkenim) var.
2. Bu kutunun içine **SADECE** bir fonksiyon koyabilirsin.
3. Ama her fonksiyonu değil! Koyacağın fonksiyon iki tane sayı parametresi almalı ve geriye mutlaka bir sayı döndürmeli.

Şimdi bu değişkeni kullanalım:

```ts
islem = topla; // (Az önce yazdığımız topla fonksiyonu) -> Hata vermez.
islem = carp;  // -> Hata vermez.

islem = selamla; // -> HATA! (Çünkü selamla string alıyordu, void dönüyordu)
```

Listing 28

Bu yapı, özellikle React'te `prop` olarak fonksiyon geçerken veya NestJS'te karmaşık servisler yazarken "***Bana şu formatta bir iş mantığı ver***" demek için kullanılır.

1.27.17. Callback Fonksiyonları Tipleme

Bunu nerede kullanacağız? En çok Callback (Geri Çağırma) fonksiyonlarında.

Diyelim ki bir fonksiyonumuz var, veritabanından veri çekiyor ve işlem bitince bize haber veriyor.

```ts
function veriGetir(id: number, callback: (veri: string) => void) {
    // Veritabanına gidildiğini hayal edin...
    console.log("Veri getiriliyor...");

    // İşlem bitti, callback'i çalıştırıyoruz
    callback("Ürün A");
}
```

Listing 29

Parametreye dikkat edin: `callback: (veri: string) ⇒ void` . Burada diyoruz ki: "***Bana ikinci parametre olarak bir fonksiyon ver. Ben işim bitince senin o fonksiyonuna bir string (veri) göndereceğim, sen ne yaparsan yap (void).***"

Kullanımı da şöyle:

```ts
veriGetir(10, (gelenVeri) => {
    console.log("İşlem tamam: " + gelenVeri);
});
```

Listing 30

Burası biraz soyut gelebilir ama Node.js dünyası callback'ler üzerine kuruludur. TypeScript ile bu callback'lerin imzasını (signature) koruma altına alırız.

1.27.18. Birleşim Tipleri (Union Types)

Konuyu değiştiriyorum. Size gerçek hayattan bir senaryo.

Bir fonksiyonunuz var, ürün **ID**'si istiyor. Ama veritabanınız biraz karışık. Eski ürünlerin **ID**'si sayı `(101, 102)` , yeni ürünlerin ID'si ise **UUID** dediğimiz metinler `("abc-123")` .

Fonksiyon parametresine `id: number` yazsanız string'ler patlar. `id: string` yazsanız sayılar patlar. `any` yazsanız... Hayır, `any` yasak demiştik.

Çözüm: **Union Types (Birleşim Tipleri)**. Sembolümüz: `|` **(Pipe - Düz çizgi)**.

```ts
function idYazdir(id: number | string) {
    console.log("Ürün ID: " + id);
}
```

Listing 31

Bu ne demek? "*Bu kapıdan içeri* `Number` *da girebilir,* `String` *de girebilir. İkisine de kapımız açık. Ama* `Boolean` *giremez!*"

`idYazdir(500);` -> Çalışır.
`idYazdir("A-500");` -> Çalışır.
`idYazdir(true);` -> Hata verir."

1.27.19. Tip Daraltma (Type Narrowing)

"Ancak Union Types kullanırken dikkat etmeniz gereken çok ince bir nokta var.

Az önceki fonksiyonun içine şunu yazmaya çalışın:

```ts
function idIsle(id: number | string) {
    // id.toUpperCase(); -> HATA!
}
```

Listing 32

Neden hata veriyor? Çünkü id bir sayı da olabilir. Sayıların büyük harfe çevrilme (`toUpperCase`) özelliği yoktur. TypeScript der ki: "*Emin değilim. Ya sayı gelirse? O yüzden bu metoda izin vermem.*"

Burada devreye **Type Narrowing (Tip Daraltma)** girer. TypeScript'e yol göstermemiz gerekir:

```ts
if (typeof id === "string") {
```

```
    // TypeScript artık burada id'nin string olduğundan %100 emin.
    console.log(id.toUpperCase());
} else {
    // Buradaysa kesinlikle number'dır.
    console.log(id.toFixed(2));
}
```

Listing 33

1.27.20. Literal Types (Tam Değer Tipleri)

Az önce **Union Types** ile "*Ya sayı olsun ya metin olsun*" dedik. Peki ya daha spesifik olmak istersek?

Mesela bir değişkenim var, adı `yon` . Bu değişken sadece ve sadece `'Kuzey'` , `'Güney'`, `'Doğu'` veya `'Batı'` değerlerini alabilsin. `'Kuzeybatı'` bile alamasın. Sadece bu 4 kelime.

Buna **Literal Types** diyoruz. Yani tip olarak `string` demek yerine, string'in kendisini yazıyoruz.

Kodlayalım:

```ts
let ruzgarYonu: "Kuzey" | "Guney" | "Dogu" | "Bati";

ruzgarYonu = "Kuzey"; // Harika, kabul etti.
// ruzgarYonu = "Yukarı"; // HATA!
```

Listing 34

TypeScript burada şunu diyor: "*Arkadaşım bu kutuya* `string` *girebilir ama* ***HER*** `string` *giremez. Sadece benim izin verdiğim o 4 kelimeden biri girebilir.*"

Bu özellik, Enum'lara çok benzer ama daha hafiftir. Genelde ayar (config) dosyalarında veya basit durum yönetimlerinde Enum yerine bunu tercih ederiz.

1.27.21. Type Aliases (Tip Takma Adları)

Şimdi bir duralım ve kodumuza bakalım.

Diyelim ki projenizde **'Kullanıcı ID'**si' diye bir kavram var. Ve bu **ID** bazen `number` , bazen `string` olabiliyor.

Fonksiyon A'da yazdınız: `id: number | string`
Fonksiyon B'de yazdınız: `id: number | string`
Fonksiyon C'de...

Sürekli aynı şeyi yazıp duruyoruz. Yarın öbür gün "*Artık ID sadece string olacak*" desek, 50 yeri değiştirmemiz gerekecek.

İşte burada **Type Alias (Tip Takma Adı)** devreye giriyor. `type` anahtar kelimesiyle kendi özel tipimizi oluşturuyoruz.

Şunu yazın:

```ts
type KimlikNo = number | string; // Kendi tipimizi icat ettik!

let calisanId: KimlikNo = 123;
let musteriId: KimlikNo = "ABC-99";
```

Listing 35

Nasıl? Çok daha temiz değil mi? Artık `number | string` yazmak yok, `KimlikNo` yazmak var. Değişiklik gerekirse sadece tek bir satırı, tanım satırını değiştirmemiz yetecek.

1.27.22. Objeler için Type Alias

Bu `type` anahtar kelimesi sadece basit tipler için değil, karmaşık Objeler için de hayat kurtarıcıdır.

Geçen dersin sonunda o upuzun, çirkin obje tanımlamasını hatırlıyor musunuz? `{ ad: string; yas: number; ... }` diye giden?

Hadi onu Type **Alias** ile paketleyelim:

```ts
type Kullanici = {
    ad: string;
    yas: number;
    email?: string; // Opsiyonel de yapabiliriz!
};
```

Listing 36

Bunu tanımladığınız anda, artık Kullanici adında bir kalıbınız var demektir.

Kullanımı:

```ts
const ahmet: Kullanici = {
    ad: "Ahmet",
    yas: 30
};

const ayse: Kullanici = {
    ad: "Ayşe",
    yas: 25,
    email: "ayse@test.com"
};
```

Listing 37

Artık her obje oluşturduğumuzda tipleri tek tek yazmıyoruz. "*Bu bir Kullanıcı'dır*" diyoruz, TypeScript gidip o kalıba uyup uymadığını kontrol ediyor.

1.27.23. Type vs Interface (Kısa Giriş)

Burada uyanık arkadaşlarımın aklına şu soru gelebilir: "*Hocam biz Java'dan/C#'tan geliyoruz, orada Interface vardı, bu ona benziyor?*"

Evet, çok benziyor! TypeScript'te objelerin şeklini tanımlamak için iki yolumuz var:

1. Type Alias (`type User = ...`)
2. Interface (`interface User ...`)

Şu an type ile başladık çünkü **Union** (`|`) işlemleri için `type` zorunludur. Ancak nesne tabanlı mimarilerde ve özellikle NestJS servislerinde Interface kullanmayı daha çok seveceğiz.

1.27.24. Interface Tanımlama

Evet arkadaşlar, type ile obje tanımlamayı gördük. Şimdi sahneye **Interface** çıkıyor.

NestJS dokümantasyonunu açtığınızda, servislerin veya veri modellerinin hep interface ile tanımlandığını göreceksiniz. Peki nedir bu?

Interface, bir objenin "*Şeklini*" (Shape) tanımlayan bir sözleşmedir.

Hadi az önceki `Kullanici` örneğini bu sefer Interface ile yazalım. `app.ts` dosyanızda şunu oluşturun:

```ts
interface Kisi {
    ad: string;
    yas: number;
    ehliyetVarMi: boolean;
}
```

Listing 38

Dikkat ettiniz mi? `type Kisi = ...` demedik. Eşittir işareti **YOK**. Doğrudan süslü parantez açtık.

Kullanımı type ile birebir aynıdır:

```ts
let musteri: Kisi = {
    ad: "Zeynep",
    yas: 28,
    ehliyetVarMi: true
};
```

Listing 39

TypeScript burada polislik yapar: '*Sen bu objeye Kisi dedin. Bakıyorum... `ad` var, `yas` var, `ehliyet` var. Tamam, sözleşmeye uymuşsun. Geçebilirsin.*'"

1.27.25. Kalıtım (Extends) ve Kod Tekrarını Önleme

Peki hocam, `type` varken neden `interface` kullanalım? Fark ne?

En büyük fark ve güç: **Kalıtım (Inheritance)**.

Diyelim ki `Kisi` diye bir temel yapımız var. Bir de `Calisan` diye bir yapımız olacak. `Calisan` dediğimiz kişi de aslında bir insandır, yani adı ve yaşı vardır. Ama ek olarak maas ve departman bilgisi vardır.

Bunu interface ile şöyle yaparız:

```ts
interface Calisan extends Kisi {
    maas: number;
    departman: string;
}
```

Listing 40

Bakın `extends Kisi` dedik. Bu şu anlama geliyor: "*Calisan interface'i, Kisi interface'inin içindeki her şeyi (ad, yas, ehliyet) miras alır, üzerine de maaş ve departmanı ekler.*"

Artık Calisan tipinde bir obje oluştururken ad bilgisini girmek zorundasınız, yoksa kızar.

```ts
const yeniPersonel: Calisan = {
    ad: "Mehmet",       // Kisi'den geldi
    yas: 35,            // Kisi'den geldi
    ehliyetVarMi: true, // Kisi'den geldi
    maas: 50000,        // Calisan'dan geldi
    departman: "IT"     // Calisan'dan geldi
};
```

Listing 41

Bu yapı, büyük NestJS projelerinde veri modellerini yönetirken hayat kurtarır.

1.27.26. Declaration Merging (Beyan Birleştirme)

Interface'lerin type'larda olmayan çok ilginç, bazen *büyülü* gelen bir özelliği daha vardır: **Merging (Birleştirme)**.

Aynı isimde iki tane type tanımlarsanız TypeScript size hata verir: **Duplicate identifier (Tekrarlanan tanımlama)** der.

Ama interface öyle değildir. Deneyelim:

```ts
interface Araba {
```

```
    marka: string;
}

// Kodun başka bir yerinde (veya aynı dosyada) tekrar tanımlıyorum:
interface Araba {
    model: string;
}
```

Listing 42

Hata verdi mi? Vermedi. Peki ne yaptı? Bu iki parçayı arka planda **BİRLEŞTİRDİ**.

Artık `Araba` tipinde bir değişken tanımlarsanız, hem `marka` hem de `model` istemek zorundadır.

Bu özellik neden var? Çünkü biz ileride başkalarının yazdığı kütüphaneleri (mesela Express.js'in Request objesini) genişletmek isteyeceğiz. Adamın koduna gidip değiştiremeyeceğimiz için, kendi dosyamızda aynı isimle interface açıp ekleme yapacağız.

1.27.27. Type vs Interface: Hangisini Seçelim?
Kafalar karıştı biliyorum. "*Hocam ne zaman* `type` *, ne zaman* `interface` *?*"

Size altın kuralı veriyorum, not alın:

1. Eğer tanımladığınız şey bir **Obje** ise veya bir **Sınıfın (Class)** taslağı ise **Interface** kullanın. (NestJS DTO'ları, Veritabanı Modelleri vb.)
2. Eğer tanımladığınız şey bir **Fonksiyon**, bir **Union (string | number)** veya bir **Tuple** ise **Type** kullanın.

Biz bu eğitimde NestJS yazacağımız için %80 oranında **Interface** (ve ileride **Class**) kullanacağız. Ama **Union Type** lazım olduğunda (`type ID = string | number`) mecbur type kullanacağız.

İkisini de bilin, ama objeler için eliniz `interface` yazmaya alışsın.

1.27.28. Sınıflara (Classes) Giriş
Arkadaşlar, **Interface** harikadır ama tek başına "*cansızdır*". Sadece bir **sözleşmedir**, kağıt üzerinde duran bir plandır.

Bize bu planı alıp, üzerinde fonksiyonların çalıştığı, iş mantığının döndüğü gerçek fabrikalar lazım. İşte buna **Class (Sınıf)** diyoruz.

NestJS'e başladığımızda göreceksiniz; yazdığımız her **Controller**, her **Service** aslında birer **sınıftır**. O yüzden burayı çok iyi anlamamız lazım.

Hemen `app.ts` 'te bir sınıf tanımlayalım:

```ts
class Araba {
    marka: string;
    hiz: number;
}
```

Listing 43

Bunu yazar yazmaz TypeScript size kızabilir: "*Property* `marka` *has no initializer.*" Diyor ki: "*Sen* `marka` *diye bir özellik koydun ama buna başlangıç değeri vermedin. Ya* `undefined` *kalırsa?*"

Bunu çözmek için sınıfın kurulum aşamasını, yani **Constructor**'ı yazmamız gerek.

1.27.29. Constructor (Kurucu Metot)

Constructor, bir sınıftan yeni bir nesne üretildiği anda (doğum anında) bir kez çalışan özel bir fonksiyondur.

Sınıfımızı şöyle güncelleyelim:

```ts
class Araba {
    marka: string;
    hiz: number;

    constructor(gelenMarka: string, gelenHiz: number) {
        this.marka = gelenMarka;
        this.hiz = gelenHiz;
    }
}
```

Listing 44

Buradaki `this` anahtar kelimesi çok kritiktir. `this.marka` -> Sınıfın kendi içindeki özellik. `gelenMarka` -> Dışarıdan gönderilen parametre.

Biz burada diyoruz ki: "***Sınıf yaratılırken gelen veriyi al, benim içimdeki özelliğe yapıştır.***"

1.27.30. Metot Ekleme (Davranış)

Sınıfları sadece veri tutmak için kullanmayız (Onun için Interface yetiyor zaten). Sınıflar eylem yapar.

Hadi arabamıza bir yetenek ekleyelim. `constructor` bloğunun altına inin (ama `class` parantezi bitmeden):

```ts
class Araba {
    marka: string;
    hiz: number;

```

```
    constructor(gelenMarka: string, gelenHiz: number) {
        this.marka = gelenMarka;
        this.hiz = gelenHiz;
    }
    hizlan(miktar: number): void {
        this.hiz = this.hiz + miktar;
        console.log(this.marka + " hızlandı. Yeni hız: " + this.hiz);
    }
}
```

Listing 45

Tıpkı normal fonksiyon yazar gibi yazdık ama başına `function` koymadık. Sınıf içinde buna Metot denir.

Artık bu araba sadece veriden ibaret değil; hızlanabilen, durumu değişebilen canlı bir nesne oldu.

1.27.31. Sınıfı Kullanma (Instance Oluşturma)
Fabrikamızı (Class) kurduk. Şimdi üretim bandından ilk arabamızı çıkaralım.

Kodun en altına inin:

```ts
class Araba {
    marka: string;
    hiz: number;

    constructor(gelenMarka: string, gelenHiz: number) {
        this.marka = gelenMarka;
        this.hiz = gelenHiz;
    }
    hizlan(miktar: number): void {
        this.hiz = this.hiz + miktar;
        console.log(this.marka + " hızlandı. Yeni hız: " + this.hiz);
    }
}

// 'new' anahtar kelimesi ile örneğini oluşturuyoruz
const benimArabam = new Araba("Tesla", 0);

console.log(benimArabam.marka); // Çıktı: Tesla

// Metodu çalıştıralım
benimArabam.hizlan(50); // Çıktı: Tesla hızlandı. Yeni hız: 50
benimArabam.hizlan(20); // Çıktı: Tesla hızlandı. Yeni hız: 70
```

Listing 46

İşte OOP (Nesne Yönelimli Programlama) budur arkadaşlar. Veriyi (hiz) ve o veriyi değiştiren kodu (hizlan) tek bir pakette topladık. NestJS'te de yapacağımız şey tam olarak bu olacak: Veritabanı bağlantısını ve sorgu kodlarını tek bir **Class** içinde paketleyeceğiz.

1.27.32. Erişim Belirleyiciler (Access Modifiers)

Sınıfımızı yazdık ama bir güvenlik açığımız var arkadaşlar.

Şu an kodun herhangi bir yerinde şunu yapabilirim: `benimArabam.hiz = 5000;`

Hız bir anda 5000 oldu! Arabanın motoru patlar. Dışarıdan birinin kafasına göre benim iç mekanizmamdaki veriyi değiştirmesini istemem. Buna yazılımda **Encapsulation (Kapsülleme)** diyoruz.

Bunu engellemek için **Access Modifier (Erişim Belirleyici)** kullanacağız. 3 tanedir:

1. **Public**: Herkese açık (Varsayılan budur).
2. **Private**: Sadece sınıfın kendi içinden erişilebilir. Dışarıya kapalıdır.
3. **Protected**: Sadece sınıfın kendisi ve ondan türeyen (miras alan) sınıflar erişebilir.

Hadi hiz özelliğini korumaya alalım. Sınıfın içine gidin:

```ts
class Araba {
    public marka: string; // Herkes okuyabilsin
    private hiz: number;  // DIŞARIYA YASAK!

    constructor(gelenMarka: string, gelenHiz: number) {
        this.marka = gelenMarka;
        this.hiz = gelenHiz;
    }
    hizlan(miktar: number): void {
        this.hiz = this.hiz + miktar;
        console.log(this.marka + " hızlandı. Yeni hız: " + this.hiz);
    }
}

// 'new' anahtar kelimesi ile örneğini oluşturuyoruz
const benimArabam = new Araba("Tesla", 0);

console.log(benimArabam.marka); // Çıktı: Tesla

// Metodu çalıştıralım
benimArabam.hizlan(50); // Çıktı: Tesla hızlandı. Yeni hız: 50
benimArabam.hizlan(20); // Çıktı: Tesla hızlandı. Yeni hız: 70
```

Listing 47

Bunu yaptığınız anda, aşağıdaki `benimArabam.hiz` kodunun altı kırmızıyla çizilir. TypeScript der ki: *"Hop! Bu özellik private. Buna dışarıdan dokunamazsın."*

1.27.33. Private'ın Mantığı

Peki hocam, `private` yaptık, dışarıdan değiştiremiyoruz. Peki nasıl hızlanacak bu araba?

Cevap: Metotlar sayesinde.

Hatırlarsanız `hizlan` metodumuz vardı ve `this.hiz` diyerek hızı artırıyordu. `hizlan` metodu sınıfın içinde olduğu için, `private` olan `hiz` değişkenine erişme hakkı vardır.

Yani dışarıdaki adam doğrudan motora müdahale edemez (`benimArabam.hiz = 100`), ama gaz pedalına basabilir (`benimArabam.hizlan(20)`). Gaz pedalı (metot), motoru (private veri) güvenli bir şekilde yönetir.

Mantığı anladınız mı? Veriyi gizle, onu yönetecek metodu dışarı aç.

1.27.34. Constructor Kısayolu (NestJS İçin Çok Kritik!)

Şimdi dersin son ve en önemli numarasını gösteriyorum. Lütfen burayı çok dikkatli dinleyin, çünkü NestJS'te yazdığımız her serviste bunu kullanacağız.

Az önce sınıfı yazarken çok fazla tekrar yaptık:

1. Yukarıda `marka` diye tanımla.
2. Constructor parametresine `gelenMarka` yaz.
3. İçeride `this.marka = gelenMarka` diye eşle.

Bu bir angarya. TypeScript'te bunun **Parameter Properties** denen muhteşem bir kısayolu var.

Sınıfı komple silin ve şu şekilde yeniden yazın:

constructor(public marka: string, private hiz: number) { }

```ts
class Araba {
  constructor(public marka: string, private hiz: number) {
    // İçeride this.marka = marka demeye gerek YOK!
    // Boş bırakabilirsiniz.
  }
  hizlan(miktar: number): void {
    this.hiz = this.hiz + miktar;
    console.log(this.marka + " hızlandı. Yeni hız: " + this.hiz);
  }
}

// 'new' anahtar kelimesi ile örneğini oluşturuyoruz
const benimArabam = new Araba("Tesla", 0);
```

```
console.log(benimArabam.marka); // Çıktı: Tesla

// Metodu çalıştıralım
benimArabam.hizlan(50); // Çıktı: Tesla hızlandı. Yeni hız: 50
benimArabam.hizlan(20); // Çıktı: Tesla hızlandı. Yeni hız: 70
```

Listing 48

Büyüye bakın! Constructor parametresinin başına `public` veya `private` yazdığınız anda TypeScript şunu anlıyor: *"Ha, sen hem parametre istiyorsun, hem de aynı isimde bir özellik (property) oluşturup onu otomatik eşlemek istiyorsun. Tamam, ben hallederim."*

Bu yazım şekli kodunuzu %50 kısaltır. NestJS'te **Dependency Injection** yaparken sadece ve sadece bu yöntemi kullanacağız.

1.28. 3. Ders

1.28.1. Generics (Jenerikler).

NestJS yazarken veya herhangi bir kütüphanenin kaynak kodunu incelerken sürekli şöyle şeyler göreceksiniz: `Repository<User>` , `Promise<boolean>` , `Partial<CreateDto>` ... Bu açılıp kapanan üçgen parantezler `< >` her yerde karşınıza çıkacak. İşte bu derste o üçgenlerin sırrını çözeceğiz. Onlar korkutucu matematik sembolleri değil, bizim en büyük yardımcılarımız.

1.28.2. Problem: 'Tekrarlayan Kod' veya 'Any Tuzağı'

Generics'in ne olduğunu anlamak için önce neden ihtiyacımız olduğunu görmemiz lazım.

Hadi basit bir senaryo düşünelim. Bana öyle bir fonksiyon yazın ki, içine ne verirsem onu bana bir dizi (`array`) içine koyup geri versin.

Eğer sayı verirsem sayı dizisi dönsün: `5 → [5]` Eğer metin verirsem metin dizisi dönsün: `"elma" → ["elma"]`

Eski bilgilerimizle bunu nasıl yazarız?

```ts
function sayiPaketle(deger: number): number[] {
  return [deger];
}
function yaziPaketle(deger: string): string[] {
  return [deger];
}
```

Listing 49

Bu çok kötü, değil mi? Her tip için ayrı fonksiyon mu yazacağız? `boolean` için ayrı, `User` class'ı için ayrı... Kod tekrarından boğuluruz.

1.28.3. `any` Kullanırsak Ne Olur?

" 'Hocam any kullanalım!' diyenleri duyar gibiyim. Hadi kullanalım, bakalım başımıza ne geliyor.

```ts
function paketle(deger: any): any[] {
  return [deger];
}
```

Listing 50

Şimdi bu fonksiyon çalışır mı? Çalışır. Ama sorun şu:

```ts
const sonuc = paketle("Merhaba");
// sonuc değişkeninin tipi ne? -> any[]
```

Listing 51

TypeScript, `sonuc` değişkeninin içinde string mi var, sayı mı var, bomba mı var bilmiyor. Yani `sonuc[0].toFixed(2)` yazsam (ki `string` 'de bu metod yok), editör beni uyarmaz. Kod çalışınca patlar.

Yani `any` kullanarak `Tip Güvenliğini (Type Safety)` kaybettik. Hem esnek olsun hem güvenli olsun istiyoruz. Mümkün mü?

1.28.4. Çözüm: `<T>` ile Generic Tanımlama

İşte burada devreye **Generics** giriyor.

Mantık şu: Fonksiyonu yazarken tipi belirlemiyoruz. Tipi, fonksiyonu çağıran kişi belirlesin diyoruz. Tıpkı bir parametre gibi, ama değer değil Tip Parametresi.

Genelde Type kelimesinin baş harfi olan T harfini kullanırız.

Hadi o fonksiyonu Generic olarak yazalım. `app.ts` dosyanızı temizleyip şunu yazın:

```ts
// <T> -> "Bana bir T tipi verileceğini biliyorum"
// deger: T -> "Gelen parametre o T tipinde olacak"
// : T[] -> "Geriye o T tipinden oluşan bir dizi döneceğim"

function paketle<T>(deger: T): T[] {
  return [deger];
}
```

Listing 52

Bu sözdizimi (syntax) başta garip gelebilir. Ama okunuşu şudur: "*Bu fonksiyon bir* `T` *tipiyle çalışır. İçeri* `T` *alır, dışarı* `T` *dizisi verir.*" Hadi kullanalım:

```ts
const sayilar = paketle<number>(10);
```

```
// sayilar değişkeninin tipi -> number[] (TypeScript artık biliyor!)

const isimler = paketle<string>("Ahmet");
// isimler değişkeninin tipi -> string[]
```

Listing 53

Hatta TypeScript o kadar zekidir ki, `<number>` yazmasanız bile içine 10 yazdığınız için `T` 'nin `number` olması gerektiğini anlar.

İşte arkadaşlar, NestJS'te veritabanından veri çekerken `repository.findOne<User>( ... )` dediğimizde, bize `User` tipinde veri dönmesinin sebebi tam olarak bu Generic yapıdır.

1.29. 3. Ders

1.29.1. Generic Interface Kullanımı (API Response Örneği)

Generic fonksiyonlar cepte. Şimdi size NestJS'te her gün kullanacağımız bir senaryo çizeyim.

Bir API yazdığınızı düşünün. Kullanıcıyı getiriyorsunuz, ürünleri getiriyorsunuz, siparişleri getiriyorsunuz... Her seferinde backend'den dönen cevap standarttır:

1. `data` : (Asıl veri burada)
2. `status` : 200 (Başarılı)
3. `message` : 'İşlem Başarılı'

Eğer Generic kullanmazsak, her veri tipi için ayrı interface yazmak zorunda kalırız: `UserResponse` , `ProductResponse` , `OrderResponse` ... Bu tam bir amelelik!

Bunun yerine Tek Bir Kalıp yapalım. `app.ts` 'e şunu yazın:

```ts
interface ApiResponse<T> {
  data: T;       // Değişen tek yer burası!
   status: number;
   message: string;
}
```

Listing 54

Bakın interface `ApiResponse<T>` dedik. "*Bu interface bir* `T` *tipiyle çalışır*" dedik.

Kullanımı nasıl?

```ts
// Kullanıcı için cevap
const userResp: ApiResponse<{ ad: string }> = {
  data: { ad: "Ahmet" }, // data, verdiğimiz tipte olmak zorunda
```

```
  status: 200,
  message: "OK"
};

// Sayı listesi için cevap
const numberResp: ApiResponse<number[]> = {
  data: [1, 2, 3], // data, artık sayı dizisi
  status: 200,
  message: "OK"
};
```

Listing 55

Gördünüz mü? Tek bir interface yazdık, binlerce farklı veri tipi için kullandık.

1.29.2. Generic Classes (Veri Deposu Örneği)

Sadece interface değil, Sınıflar (Class) da Generic olabilir.

Hadi basit bir Veri Deposu (`Storage`) sınıfı yazalım. Bu sınıfın içine veri ekleyelim (`add`) ve verileri çekelim (`getAll`).

Ama bu depo her şeyi saklayabilsin. İster sayı saklasın, ister metin.

```ts
class Depo<T> {
  private veriler: T[] = []; // T tipinden oluşan bir dizi

  ekle(yeniVeri: T): void {
      this.veriler.push(yeniVeri);
  }

  hepsiniGetir(): T[] {
      return this.veriler;
  }
}
// Metin Deposu yapalım
const metinDeposu = new Depo<string>();
metinDeposu.ekle("Not 1");
metinDeposu.ekle("Not 2");
// metinDeposu.ekle(100); -> HATA! Çünkü kutunun üzerine 'String' yazdık.

// Sayı Deposu yapalım
const sayiDeposu = new Depo<number>();
sayiDeposu.ekle(50);
```

Listing 56

Bu yapı sayesinde, kodunuzu **Type-Safe (Tip Güvenli)** tutarak son derece esnek hale getirebilirsiniz.

1.29.3. Constraints (Kısıtlamalar - `extends`)

Buraya kadar her şey harika. `<T>` dedik, " `ne verirsen kabulümdür` " dedik. Ama bazen " `Her şeyi kabul etmem, sadece belli şartları sağlayanları kabul ederim` " demek isteriz.

Mesela bir fonksiyon yazacağız, gelen objenin idsini ekrana basacak.

```ts
function idYazdir<T>(item: T) {
  console.log(item.id); // HATA!
}
```

Listing 57

TypeScript burada bas bas bağırır: "***Kardeşim sen `T` dedin, bu `T` her şey olabilir. Ya bana sadece sayı (number) gelirse? Sayıların id'si olmaz ki!***"

Haklı. Bizim TypeScript'e şunu dememiz lazım: 'Bak T, sen her şey olabilirsin AMA içinde mutlaka bir id alanı olmak zorunda.'

Buna **Generic Constraint (Kısıtlama)** denir ve `extends` anahtar kelimesiyle yapılır. Düzeltelim:

```ts
// Şartımızı bir interface olarak tanımlayalım
interface Kimlikli {
  id: number;
}

// T extends Kimlikli -> T, Kimlikli interface'ini miras almak ZORUNDA
function idYazdir<T extends Kimlikli>(item: T) {
  console.log(item.id); // Artık hata yok!
}
```

Listing 58

Kullanımı: `idYazdir({ id: 1, ad: "Ahmet" });` -> Çalışır. `idYazdir({ ad: "Mehmet" });` -> HATA! (id eksik).

Bu özellik, NestJS'te veritabanı sorguları atarken "***Sadece veritabanı tablosu olan sınıfları kabul et' demek için çok işimize yarayacak.***"

1.29.4. Decorators (Dekoratörler) Giriş

Arkadaşlar, Generics ile esnekliği sağladık. Şimdi size NestJS'in o 'Büyülü' dünyasının kapılarını açıyorum.

NestJS kodu gördüyseniz, her yerin `@` (At işareti) ile başlayan kelimelerle dolu olduğunu fark etmişsinizdir. `@Controller` , `@Get` , `@Post` , `@Body` , `@Injectable` ...

Bunlara **Decorator (Dekoratör)** diyoruz.

Peki nedir bu Dekoratör? Adı üzerinde, kodumuzu "*dekore eden*", onu süsleyen yapılar. Ama sadece süslemekle kalmazlar, kodumuza **Metadata (Üst veri)** eklerler ve davranışını değiştirirler.

Bir sınıfın üzerine `@Controller` yazdığınızda, NestJS o sınıfa bakar ve der ki: "*Hımm, demek ki bu sıradan bir sınıf değil, bu web isteklerini karşılayacak bir* `Router` *sınıfı.*" İşte bu etiketi yapıştıran şey **Dekoratördür**.

1.29.5. Decorator Nedir? (Teknik Tanım)

Teknik olarak konuşursak; Decorator aslında basit bir fonksiyondur. Evet, bildiğimiz fonksiyon.

Ama özel bir fonksiyondur. Bir sınıfın, bir metodun veya bir özelliğin başına `@` işaretiyle koyduğunuzda, TypeScript otomatik olarak o fonksiyonu çalıştırır ve içine sizin sınıfınızı/metodunuzu parametre olarak gönderir.

Yani siz:

```ts
@OzelBirSey
class Araba { ... }
```

Listing 59

yazdığınızda, arka planda aslında şu olur: `OzelBirSey(Araba)`

Bu fonksiyon, sınıfı alır ve istenene özellikleri içine injekte eder, loglar, değiştirir ve geri koyar. Biz buna **Meta-Programming** diyoruz.

1.29.6. Ayar: `experimentalDecorators`

Yalnız, Decorator'ları kullanabilmek için TypeScript'e özel bir izin vermemiz lazım. Çünkü bu özellik JavaScript'in standartlarında henüz tam oturmuş bir özellik değil (Hala deneysel sayılır).

Hemen `tsconfig.json` dosyanızı açın. Orada arama yapın (CTRL+F): `experimentalDecorators` .

Bu satırın başındaki yorum işaretlerini (//) kaldırın ve değerini `true` yapın. Eğer bu ayarı açmazsanız, `@` işareti koyduğunuz her yerde hata alırsınız.

Ayrıca `emitDecoratorMetadata` ayarını da bulup `true` yaparsanız harika olur, NestJS bunu da çok sever.

1.29.7. İlk Class Decorator'ımızı Yazıyoruz

Ayarları kaydettiyseniz, `app.ts` dosyamızı temizleyelim ve ilk dekoratörümüzü yazalım.

Senaryomuz şu: Öyle bir dekoratör yazalım ki, hangi sınıfın tepesine koyarsak, o sınıf oluşturulduğunda konsola "***Bu sınıf loglandı!***" yazsın.

Kodlayalım:

```ts
// Dekoratör Fonksiyonu (Baş harfi genelde büyük olur)
function Logla(target: Function) {
  console.log("Logla dekoratörü çalıştı!");
  console.log("Hedef sınıf: " + target.name);
}
// Kullanımı
@Logla
class Personel {
  constructor() {
    console.log("Personel sınıfı oluşturuluyor...");
  }
}
```

Listing 60

Bunu yazıp `ts-node app.ts` ile çalıştırdığınızda ilginç bir şey göreceksiniz. Biz `new Personel()` demedik. Sınıfı çağırmadık bile. Ama konsolda "***Logla dekoratörü çalıştı!***" yazısını göreceksiniz.

Neden? Çünkü kod derlenirken **(Compile time)** TypeScript o `@Logla` işaretini gördü ve fonksiyonu hemen çalıştırdı.

1.29.8. Decorator Factories (Parametre Alan Dekoratörler)

Az önce `@Logla` yazdık, bitti. Hiçbir parametre vermedik. Ama NestJS'te `@Controller('users')` veya `@Get('profile')` gibi parantez açıp içine bir şeyler yazıyoruz. Bizim dekoratörümüz neden parametre almıyor?

Çünkü dekoratörler standart olarak sadece hedef sınıfı (target) parametre alır. Eğer biz dışarıdan `'Merhaba'` , `'Admin'` , `'User'` gibi özel bilgiler göndermek istiyorsak, **Decorator Factory (Dekoratör Fabrikası)** desenini kullanmamız lazım.

Korkmayın, ismi havalı ama kendisi çok basit: **İç içe fonksiyon.**

```ts
function Logla(mesaj: string) {
  // Dışarıdaki fonksiyon parametreyi (mesaj) alır
  return function (target: Function) {
    // İçerideki fonksiyon asıl işi yapar
    console.log(mesaj);
    console.log(target);
  }
}
```

```
@Logla("Sistem Başlatılıyor...")
class Server {
  // ...
}
```

Listing 61

Gördünüz mü? Artık `@Logla` değil, `@Logla( ... )` şeklinde çağırıyoruz. NestJS'in bütün sırrı bu **Factory** yapısında saklıdır."

1.29.9. Method Decorators (Metot Dekoratörleri)

Sınıfın tepesine yazdıklarımız cepte. Peki ya metodların tepesine yazdıklarımız? Yani `@Get` , `@Post` gibi olanlar?

Bunlara **Method Decorator** denir. Bir metot dekoratörü yazdığınızda, TypeScript bu fonksiyona üç tane parametre gönderir. Bu parametreleri ezberlemenize gerek yok ama ne işe yaradıklarını bilin:

1. `target` : Sınıfın prototipi (iskeleti).
2. `propertyKey` : Metodun adı (string olarak).
3. `descriptor` : Metodun özelliklerini tutan obje (en önemlisi bu!).

Hadi bir tane yazalım.

```ts
// target, key, descriptor
function MetoduLogla(target: any, key: string, descriptor: PropertyDescriptor) {
    console.log("Metot çalıştı: " + key);
}
class HesapMakinesi {
  @MetoduLogla
  topla(a: number, b: number) {
    return a + b;
  }
}
```

Listing 62

Bunu çalıştırdığınızda, sınıf tanımlandığı anda "***Metot çalıştı: topla***" yazısını göreceksiniz. Ama asıl büyü, o metodu değiştirebildiğimizde başlar.

1.29.10. Metodu Manipüle Etmek (Descriptor.value)

Şimdi çok dikkatli dinleyin. Burası NestJS'in 'Interceptor' ve 'Guard' mantığının temelidir.

`descriptor` dediğimiz o 3. parametrenin içinde `value` diye bir özellik vardır. `descriptor.value =` Metodun ta kendisidir!

Biz bu `value` özelliğini değiştirerek, orijinal metodun yerine kendi yazdığımız fonksiyonu koyabiliriz.

Örnek: Bir metot çalışmadan önce "***Başlıyor***", bitince "***Bitti***" yazdıran bir dekoratör yapalım.

```ts
function SureyiOlc(target: any, key: string, descriptor: PropertyDescriptor) {
  const orjinalMetot = descriptor.value; // Metodun yedeğini al (topla fonksiyonu)

  // Yeni bir fonksiyon yazıp orjinalin yerine koyuyoruz
  descriptor.value = function (...args: any[]) {
    console.log("İşlem başlıyor...");

    // Orijinal metodu çalıştır (apply ile)
    const sonuc = orjinalMetot.apply(this, args);

    console.log("İşlem bitti. Sonuç: " + sonuc);
    return sonuc;
  };
}
```

Listing 63

Şimdi bu dekoratörü topla metoduna takıp çalıştırırsanız, siz `sadece topla(5, 10)` deseniz bile konsolda şunu görürsünüz:

İşlem başlıyor...

(Sonuç hesaplanır)

İşlem bitti. Sonuç: 15

İşte `AOP (Aspect Oriented Programming)` budur arkadaşlar. Kodun içine dokunmadan, dışarıdan davranış ekledik.

1.29.11. Property Decorators (Özellik Dekoratörleri)

Metotları hallettik. Peki ya sınıfın içindeki değişkenler?

NestJS'te DTO yazarken şöyle şeyler göreceksiniz:

```ts
class CreateUserDto {
    @IsString()
    ad: string;

    @IsInt()
    yas: number;
}
```

Listing 64

İşte bu değişkenlerin tepesine konan `@IsString` , `@IsInt` gibi yapılar **Property Decorator**'dır.

Bunları yazmak metot dekoratörlerine göre biraz daha basittir ama bir o kadar da gıcıktır. Çünkü Property Decorator fonksiyonu sadece 2 parametre alır:

1. **target:** Sınıfın prototipi.
2. **key:** Özelliğin adı (Örn: 'ad', 'yas').

Dikkat! Descriptor yoktur. Yani o özelliğin değerine (`value`) doğrudan erişemezsiniz. Sadece "***Burada ad diye bir özellik var***" diye işaretleme yapabilirsiniz.

1.29.12. Metadata Mantığı (Etiketleme)

Peki değerine erişemiyorsak bu dekoratör ne işe yarıyor?

Şöyle düşünün: Bir kargo şirketindesiniz. Kutuların **(Property)** içine bakamıyorsunuz ama kutunun üzerine "***KIRILACAK EŞYA***" etiketi yapıştırıyorsunuz. Daha sonra kargoyu taşıyan kişi (NestJS / Class-Validator), o etiketi görüyor ve kutuya ona göre davranıyor.

Biz Property Decorator'ları, o özelliğe **Metadata (Gizli Bilgi)** eklemek için kullanırız.

```ts
function ZorunluAlan(target: any, key: string) {
    // target'ın üzerine gizli bir not defteri açıyoruz
    // "Bu sınıfta, şu 'key' (özellik) zorunludur" diye not alıyoruz.
    console.log(key + " alanı işaretlendi.");
}

class Urun {
    @ZorunluAlan
    fiyat: number;
}
```

Listing 65

NestJS arka planda `reflect-metadata` denen bir kütüphane kullanarak bu notları okur ve "***Hımm, fiyat alanı işaretlenmiş, o zaman bunu kontrol edeyim***" der.

1.29.13. Parameter Decorators (Parametre Dekoratörleri)

Geldik dekoratör ailesinin son üyesine.

NestJS'te Controller metodlarını yazarken şöyle yapacağız:

```ts
findUser(@Param('id') id: string, @Body() data: any)
```

Listing 66

Buradaki `@Param` ve `@Body` , fonksiyonun parametrelerinin içine yazılır. Bunlara **Parameter Decorator** denir.

Bu dekoratör fonksiyonu üç parametre alır:

1. **target:** Sınıfın prototipi.
2. **methodKey:** Hangi metodun içinde olduğumuz.
3. **parameterIndex:** Kaçıncı parametre olduğu (0, 1, 2...).

Bu neden önemli? NestJS, bir HTTP isteği geldiğinde şunu bilmek ister: "***Kullanıcı bana bir veri gönderdi. Ben bu veriyi metodun 1. parametresine mi koyayım, yoksa 2. parametresine mi?***"

İşte `@Body()` dekoratörü aslında şunu yapar: "***Hey NestJS! Gelen Request'in gövdesini (Body), bu metodun 1. sırasındaki (Index 1) parametresine enjekte et.***"

Yani olay tamamen **Index (Sıra)** tutmaktır.

1.29.14. Spagetti Koddan Kurtuluş: Modülerlik

Arkadaşlar, şu ana kadar her şeyi tek bir dosyaya, `app.ts` içine yazdık. 50 satırlık kodda bu sorun değil. Ama NestJS ile gerçek bir proje yazdığımızda 50.000 satır kodumuz olacak. Hepsini tek dosyaya yazarsak ne olur? O dosyayı açmak bile 5 dakika sürer, aradığımızı bulamayız.

Yazılımda kural şudur: 'Böl ve Yönet' (Divide and Conquer).

Kodumuzu mantıklı parçalara, dosyalara ayırmalıyız.

- Kullanıcı işleri -> `user.ts`
- Ürün işleri -> `product.ts`
- Matematik işlemleri -> `math.ts`

Ama TypeScript'te bir dosyayı böldüğünüzde, o dosya varsayılan olarak dış dünyaya kapalıdır. İçindeki fonksiyonu başka dosya göremez. İşte burada **Import / Export** mekanizması devreye girer.

1.29.15. Export ve Import Temelleri

Hadi hemen deneyelim. `app.ts` 'in yanına yeni bir dosya oluşturun: `matematik.ts` .

İçine şunu yazın:

```ts
const PI = 3.14;

function daireAlani(yaricap: number) {
    return PI * yaricap * yaricap;
}
```

Listing 67

Şimdi `app.ts` dosyasına gidin ve şunu yazmaya çalışın: `console.log(daireAlani(5));`

Hata alacaksınız: "*Cannot find name daireAlani*". Neden? Çünkü `matematik.ts` dosyasındaki o fonksiyon **Private (Gizli)** durumdadır.

Dışarı açmak için fonksiyonun başına `export` kelimesini eklememiz lazım.

`matematik.ts` dosyasına dönün:

```ts
const PI = 3.14;

export function daireAlani(yaricap: number) {
    return PI * yaricap * yaricap;
}
```

Listing 68

Şimdi `app.ts` 'e dönün ve en tepeye şunu ekleyin:

```ts
import { daireAlani } from './matematik';

console.log(daireAlani(5)); // Artık çalışıyor!
```

Listing 69

İşte bu kadar. `export` kapıyı açar, `import` içeri girer.

1.29.16. Named Exports vs Default Exports

Burada iki farklı yöntemimiz var ve NestJS dünyasında hangisini tercih edeceğimiz önemli.

1. **Named Export (İsimli Dışa Aktarım):** Az önce yaptığımız gibi. `export class User ...` Bunu `import` ederken süslü parantez `{ }` kullanmak zorundasınız ve aynı ismi kullanmak zorundasınız. `import { User } from ...`
2. **Default Export (Varsayılan Dışa Aktarım):** Dosyadan sadece **TEK** bir şey dışarı çıkacaksa kullanılır. `export default class User ...` Bunu `import` ederken süslü parantez kullanılmaz ve ismini değiştirebilirsiniz. `import Kullanici from ...`

Benim Tavsiyem: NestJS geliştirirken %99 oranında **Named Export (birinci yöntem)** kullanacağız. Çünkü "*Refactoring*" yaparken (yani kodun adını değiştirirken) editörler **Named Export**'u çok daha iyi yönetir. **Default Export** bazen kafa karışıklığı yaratır.

1.29.17. Barrel Dosyaları (`index.ts`)

Modülerlikte çok havalı bir teknik daha var: **Barrel (Fıçı) Dosyaları**.

Diyelim ki **utils** diye bir klasörünüz var. İçinde 50 tane küçük dosya var. Bunları kullanmak için `app.ts` dosyasında 50 satır `import` yazmak istemezsiniz:

```ts
import { a } from './utils/a';
import { b } from './utils/b';
// ... böyle 50 tane!!
```

Listing 70

Bunun yerine utils klasörünün içine bir `index.ts` dosyası koyarız. Bu dosya bir trafik polisisidir.

```ts
export * from './a';
export * from './b';
export * from './c';
```

Listing 71 : `utils/index.ts`

Bunu yaparsak, artık `app.ts` dosyasında tek satırda işi bitiririz:

```ts
import { a, b, c } from './utils'; // Direkt klasör adı!
```

Listing 72 : `app.ts`

Node.js ve TypeScript, bir klasörü `import` ettiğinizde otomatik olarak `index.ts` dosyasına bakar. Bu teknik, projenizi inanılmaz temiz tutar.

1.29.18. Senkron vs Asenkron (Beklemek ya da Beklememek)

Arkadaşlar, JavaScript'in çok önemli bir huyu vardır: Sabırsızdır.

JavaScript (ve Node.js) tek şeritli bir otoban gibidir **(Single Thread)**. Eğer siz kodun ortasında "***Şurada 5 saniye bekle***" derseniz, arkadaki tüm trafik durur. Site donar, kimse işlem yapamaz. Buna "*Blocking Code*" **(Bloklayan Kod)** denir ve biz bunu asla istemeyiz.

Ama gerçek hayatta işler zaman alır. Veritabanına "***Kullanıcıları getir***" dediğimizde bu işlem bir saniye sürebilir. JavaScript bu durumda ne yapar? "***Tamam, sen o veriyi getire dur, ben beklemiyorum, alttaki satırları çalıştırmaya devam ediyorum***" der.

İşte buna **Asenkron Programlama** diyoruz.

Ama burada büyük bir sorun var: Eğer JavaScript beklemeden alt satıra geçerse, ve alt satırda o veriyi kullanmaya çalışırsa ne olur? Veri daha gelmediği için undefined hatası alırız ve patlarız.

1.29.19. Promise (Söz Verme) Yapısı

Bu sorunu çözmek için JavaScript bize bir **Promise (Söz)** verir.

Mantık şudur: Veritabanına gidip veriyi istediğinizde, o size veriyi hemen veremez. Ama size bir "*Söz*" **(Promise nesnesi)** verir. Der ki: "***Bak şu an elim boş ama söz veriyorum, işlem bitince sana ya veriyi getireceğim (Resolve) ya da hata fırlatacağım (Reject).***"

Eskiden (2015 civarı) bunu `.then()` ve `.catch( )` bloklarıyla yönetirdik.

```ts
// ESKİ YÖNTEM (NestJS'te pek kullanmayacağız)
veriGetir()
    .then((veri) => {
        console.log("Veri geldi: " + veri);
    })
    .catch((hata) => {
        console.log("Hata oldu: " + hata);
    });
```

Listing 73

Bu yapı çalışır ama kod büyüdükçe iç içe geçmiş merdivenlere dönüşür **(Callback Hell)**. Okuması zordur.

1.29.20. Modern Çözüm: Async / Await

İşte burada sahneye modern kahramanımız çıkıyor: **Async / Await**.

Bu yapı, asenkron kodu sanki senkronmuş (sıralıymış) gibi okumamızı sağlar.

Bir fonksiyonun içinde bekleme yapacaksanız, o fonksiyonun başına `async` kelimesini koyarsınız. Bekleyeceğiniz işlemin (`Promise` döndüren işlemin) başına da `await` koyarsınız.

Şunu yazın `app.ts` 'e:

```ts
// Yapay bir bekleme fonksiyonu (2 saniye bekler)
function bekle(ms: number): Promise<string> {
    return new Promise(resolve => setTimeout(() => resolve("İşlem Tamam!"), ms));
}

// Ana fonksiyonumuz (async olmak ZORUNDA)
async function baslat() {
    console.log("1. Başlıyor...");

    // Kod burada 2 saniye DURUR (Sadece bu fonksiyon durur, sunucu durmaz)
    const sonuc = await bekle(2000);

    console.log("2. " + sonuc);
    console.log("3. Bitti.");
}

baslat();
```

Listing 74

Bunu çalıştırdığınızda (`ts-node app.ts`), çıktının sırayla geldiğini ve arada 2 saniye beklediğini göreceksiniz. `await` kelimesi, "*Promise tamamlanana kadar alt satıra geçme*" emrini verir.

1.29.21. NestJS İçin Neden Kritik?

Hocam bu neden bu kadar önemli?

Çünkü önümüzdeki haftalarda NestJS ile veritabanı sorguları yazarken kodumuz hep şöyle olacak:

```ts
// (Örnek Kod - Yazmanıza gerek yok)
async findOne(id: number) {
    const user = await this.userRepository.findOne(id); // Veritabanını bekle!

    if (!user) {
        throw new NotFoundException(); // Yoksa hata at
    }

    return user; // Varsa dön
}
```

Listing 75

Eğer oraya `await` koymayı unutursanız, kodunuz veritabanını beklemeden çalışır ve `user` değişkeni boş olduğu için hata alırsınız. O yüzden kuralımız basit: **Veritabanı varsa, `await` vardır. `await` varsa, fonksiyon `async` olacaktır.**

1.29.22. Asenkron Hata Yönetimi (Try-Catch)

Arkadaşlar, `await` harikadır ama tehlikelidir. Eğer `await` ile beklediğiniz işlem (mesela veritabanı sorgusu) hata verirse, uygulamanız çöker.

Eskiden `.catch()` kullanıyorduk. **Async/Await** dünyasında ise klasik JavaScript **Try-Catch** bloğunu kullanırız.

Kodumuzu güvenli hale getirelim:

```ts
async function veriGetir() {
    try {
        // Hata çıkma ihtimali olan kodları buraya yazarız
        console.log("Veri isteniyor...");
        const veri = await veritabaniSorgusu();
        console.log("Veri geldi: ", veri);

    } catch (hata) {
        // Hata olursa program çökmez, buraya düşer
        console.log("Bir sorun oluştu!");
```

```
        console.log("Hata detayı: ", hata);
    }
}
```

Listing 76

Bu yapı NestJS'te çok önemlidir. Mesela kullanıcı kayıt olurken aynı email adresi veritabanında varsa, veritabanı hata fırlatır. Biz o hatayı `catch` bloğunda yakalayıp kullanıcıya "*Bu email zaten kullanılıyor*" diye güzel bir mesaj döneceğiz.

1.29.23. Async Fonksiyonların Dönüş Tipi (Promise Wrapper)

Şimdi çok teknik ve mülakatlarda sorulan bir detaya değineceğim.

Şu fonksiyona bakın:

```ts
async function topla(a: number, b: number): number { // HATA!
    return a + b;
}
```

Listing 77

Normalde `a+b` bir sayıdır. Ama fonksiyonun başında `async` yazıyorsa, bu fonksiyon `ASLA` doğrudan number döndürmez.

`async` olan her fonksiyon, sonucunu otomatik olarak bir **Promise Kutusuna sarar**. Yani siz `5` döndürseniz bile, o aslında `Promise<number>` olur.

Doğrusu şudur:

```ts
async function topla(a: number, b: number): Promise<number> {
    return a + b;
}
```

Listing 78

Bunu bir kargo paketi gibi düşünün. `async` fonksiyon size ürünü (`number`) vermez, içinde ürün olan kargo paketini (`Promise`) verir. O paketi açmak (`await`) veya paketin gelmesini beklemek (`.then`) size kalmıştır.

1.29.24. Promise.all (Paralel İşlemler)

Peki 3 farklı veriye ihtiyacımız varsa?

1. Kullanıcıyı getir (1 saniye)
2. Ürünleri getir (1 saniye)
3. Ayarları getir (1 saniye)

Eğer bunları alt alta `await` ile yazarsak; `1+1+1 = 3` saniye bekleriz. Halbuki bu işlemlerin birbirine ihtiyacı yok. Hepsi aynı anda başlayabilir.

Bunun için `Promise.all` kullanırız.

```ts
// Üçünü aynı anda başlatır ve hepsi bitene kadar bekler
const [user, urunler, ayarlar] = await Promise.all([
    getUser(),
    getProducts(),
    getSettings()
]);
```

Listing 79

Bu sayede toplam bekleme süresi **1** saniyeye düşer. NestJS'te **Dashboard (Gösterge Paneli)** yaparken performans için bunu çok sık kullanacağız.

1.30. Profesyonel Geliştirme Ortamı

1.30.1. Giriş ve "Kırık Camlar Teorisi"

Şu ana kadar TypeScript ile değişken tanımladık, fonksiyon yazdık, class kurduk. Kodumuz çalışıyor mu? Çalışıyor. Ama kodun çalışması yetmez. Kodun **Güzel** ve **Okunabilir** olması gerekir.

Yazılımda "***Kırık Camlar Teorisi***" diye bir şey vardır. Eğer projenizde bir kod satırı çirkinse, girintileri (indentation) bozuksa, değişken isimleri kötüyse; o projeye giren diğer yazılımcı da "***Aman zaten ortalık dağınık***" der ve daha da kötü kod yazar. Bir ay sonra proje çöplüğe döner.

Bu derste, kodumuzu askeri bir disipline sokacak iki ajanı projemize dahil edeceğiz:

1. **Prettier:** Kodun şeklini şemalini düzelten makyaj uzmanı.
2. **ESLint:** Kodun kalitesini denetleyen polis memuru.

1.30.2. Prettier Nedir? Neden Lazım?

İlk ajanımız: **Prettier**.

Eskiden yazılım ekiplerinde kavgalar çıkardı:

- 'Ben kodun sonunda noktalı virgül severim!'
- 'Hayır, ben sevmem!'
- 'Ben girinti için 2 boşluk bırakırım!'
- 'Ben 4 boşluk bırakırım!'

Bu kavgalar zaman kaybıdır. Prettier diyor ki: "***Siz tartışmayın. Kuralları bir kere belirleyin, ben her kaydet (CTRL+S) tuşuna bastığınızda kodu otomatik olarak o kurallara göre hizalayayım.***"

Böylece kodunuzu kim yazarsa yazsın, sanki tek bir kişinin elinden çıkmış gibi görünür.

1.30.3. Prettier Kurulumu

Hadi şu sihirli değneği kuralım. Terminalinizi açın (Proje klasöründe olduğunuzdan emin olun).

Şu komutu yazın:

```shell
npm install --save-dev prettier
```

Listing 80

Buradaki `--save-dev` (veya kısa haliyle `-D`) çok önemli. Çünkü Prettier, sadece biz geliştirme yaparken lazım. Kodumuz canlı sunucuya **(Production)** çıktığında kimse kodun şekliyle ilgilenmez, çalışmasıyla ilgilenir. O yüzden bunu "***Geliştirici Bağımlılığı***" (`devDependencies`) olarak kuruyoruz.

1.30.4. Konfigürasyon Dosyası (`.prettierrc`)

Kurulum bitti ama Prettier henüz ne yapacağını bilmiyor. Ona kurallarımızı söylememiz lazım.

Projenizin kök dizininde (`app.ts` 'in olduğu klasörde değil, en dışta, `package.json` 'ın yanında) yeni bir dosya oluşturun. Adı tam olarak şu olmalı (başındaki nokta önemli): `.prettierrc`

İçine şu kuralları yapıştırın (Bu benim önerdiğim standart NestJS setidir):

```json
{
  "singleQuote": true,
  "trailingComma": "all",
  "tabWidth": 2,
  "semi": true,
  "printWidth": 100
}
```

Listing 81

Bu kurallar ne diyor?

1. `singleQuote: true` -> 'Çift tırnak yerine tek tırnak kullan.'
2. `trailingComma: all` -> 'Objelerin son elemanından sonra da virgül koy.' (Git diff'leri için harikadır).
3. `semi: true` -> 'Her satırın sonuna noktalı virgül koy.'
4. `printWidth: 100` -> "Satır uzunluğu 100 karakteri geçerse alt satıra indir."

Kaydedin ve kapatın. Artık kurallarımız hazır.

1.30.5. VS Code Entegrasyonu (Format On Save)

Dosyayı oluşturduk ama henüz büyü gerçekleşmedi. `app.ts` 'e gidip kodu karıştırsanız bile düzelmiyor. Çünkü VS Code'a "*Prettier'ı kullan*" demedik.

Hemen VS Code'un Sol Menüsündeki Extensions (Kareler ikonu) kısmına gelin. Arama çubuğuna "*Prettier*" yazın ve "*Prettier - Code formatter*" eklentisini bulup Install (Yükle) diyerek kurun. (Çoğunuzda zaten vardır ama emin olalım).

Şimdi asıl ayarı yapıyoruz:

1. CTRL + , (Virgül) tuşlarına basarak Ayarları açın.
2. Arama çubuğuna 'Format On Save' yazın.
3. Çıkan kutucuğu işaretleyin (Check).

Bitmedi! Bir de "*Default Formatter*" ayarı var. Arama çubuğuna "*Default Formatter*" yazın ve açılan listeden **Prettier - Code formatter** seçeneğini seçin.

Şimdi `app.ts` dosyasına dönün. Kodun girintilerini bozun, saçma sapan boşluklar bırakın. Sonra `CTRL+S` yapın. Şak! Kodunuz anında hizalandı değil mi? İşte verimlilik budur.

1.30.6. İkinci Ajan: ESLint (Kod Polisi)

Prettier kodumuzu güzelleştirdi. Ama güzel görünen bir kod, "*Hatalı*" bir kod olabilir.

Mesela:

```ts
let sayi = 5; // Değişken tanımladın
console.log("Merhaba"); // Ama değişkeni hiç kullanmadın!
```

Listing 82

Prettier buna karışmaz. "*Hizalaması düzgün mü? Düzgün. Benim işim bitti*" der. Ama bu kodda bir mantık hatası var. Kullanmayacağın değişkeni neden tanımladın? Veya sonsuz döngüye girecek bir kod yazdın, Prettier bunu anlamaz.

İşte burada devreye **ESLint (ECMAScript Lint**) girer. ESLint, kodun Kalitesini ve Mantığını denetler. Hatalı kullanımı, potansiyel bug'ları ve kötü alışkanlıkları yakalar.

1.30.7. Prettier vs ESLint Farkı

Kafanızda şöyle kodlayın:

- **Prettier:** Makyaj Uzmanıdır. Kodun yüzünü güzelleştirir. (Format)
- **ESLint:** Trafik Polisidir. Kural ihlali var mı diye bakar. (Quality)

Biz projemizde ikisini birden kullanacağız. Biri güzelleştirecek, biri denetleyecek. Hatta bazen çatışacaklar ama biz onları barıştıracağız.

1.30.8. ESLint Kurulumu

Hadi şu polisi de karakola (projemize) getirelim.

TypeScript ile ESLint'i beraber kullanmak biraz çetrefillidir çünkü ESLint normalde sadece JavaScript anlar. Ona "***Sen TypeScript de anlayacaksın***" dememiz lazım.

Terminali açın ve şu uzun komutu yapıştırın (Tek satırda):

```shell
npm install --save-dev eslint @typescript-eslint/parser @typescript-eslint/eslint-plugin
```

Listing 83

Burada 3 şey kuruyoruz:

+ **eslint:** Aracın kendisi.

1. **parser:** ESLint'in TypeScript kodunu okuyabilmesini sağlayan çevirmen.
2. **plugin:** TypeScript'e özel kuralları (Best Practices) içeren kural seti.

`Enter` tuşuna basın ve kurulmasını bekleyin. Birazdan bu polise kuralları öğreteceğiz.

1.30.9. ESLint Yapılandırma Dosyası (.eslintrc)

Paketleri indirdik ama ESLint henüz devre dışı. Ona "***Git şu dosyaları denetle***" demedik.

Projenin kök dizininde (`package.json` dosyasının yanında) yeni bir dosya oluşturun: `.eslintrc` (veya `.eslintrc.json`).

İçine şu temel ayarları yapıştırın. Bu ayarlar, TypeScript dünyasının standart anayasasıdır:

```json
{
  "parser": "@typescript-eslint/parser",
  "plugins": ["@typescript-eslint/"],
  "extends": [
    "plugin:@typescript-eslint/recommended"
  ],
  "rules": {
    "no-console": "warn",
    "@typescript-eslint/no-explicit-any": "error"
  }
}
```

Listing 84

Bu ne anlama geliyor?

1. **parser:** 'Ben normal JS değil, TypeScript okuyacağım' diyor.
2. **extends:** "***Dünyaca kabul görmüş (recommended) kuralları otomatik uygula***" diyor. Tek tek 100 tane kural yazmakla uğraşmıyoruz.

3. **rules:** Burası bizim özel isteklerimiz. Mesela `no-console: warn` diyerek "*Kodun içinde* `console.log` *unutursam beni uyar (sarı yak), ama hata verme (kırmızı yakma)*" dedik.

1.30.10. Kuralları Test Etmek

Hadi çalışıyor mu bakalım!

`app.ts` dosyanızı açın. Şu yasaklı kodu yazın:

```shell
function test(veri: any) { // any kullandık!
    console.log(veri);      // console.log kullandık!
}
```

Listing 85

Eğer VS Code eklentileriniz tamamsa, `any` kelimesinin altının **KIPKIRMIZI** çizildiğini göreceksiniz. Üzerine gelince ESLint bağırır: "*Unexpected any. Specify a different type.*" (Beklenmedik any kullanımı. Başka tip belirt).

`console.log` 'un altı ise **SARI** çizilir. "*Bunu burada unutma*" der.

Gördünüz mü? Artık kodunuzu derlemeden (compile etmeden) hataları editörde görüyorsunuz. Bu size inanılmaz hız kazandırır.

1.30.11. Polisin Girmemesi Gereken Yerler (.eslintignore)

Arkadaşlar, projemizde bazı dosyalar var ki, biz yazmıyoruz. Bilgisayar otomatik üretiyor. Mesela `node_modules` klasörü veya kod derlendiğinde oluşan `dist` klasörü.

ESLint'in gidip binlerce hazır kütüphane dosyasını taramasını istemeyiz. Hem yavaşlar hem de bizim suçumuz olmayan hatalar bulur.

Kök dizine `.eslintignore` dosyası açın ve içine şunları yazın:

```shell
node_modules
dist
coverage
```

Listing 86 : .eslintignore

Bu, polise "*Bu odalara girme, orası özel mülk*" demektir.

1.30.12. Büyük Çatışma: Prettier vs ESLint

Şimdi çok kritik bir noktadayız. Prettier kodun şekline karışır, ESLint de bazen şekle karışır.

Örnek:

1. **Prettier:** "*Ben tek tırnak severim.*"
2. **ESLint:** "*Hayır, kural kitabımda çift tırnak yazıyor, hata ver!*"

Bu durumda VS Code ne yapacağını şaşırır. Kaydedersiniz Prettier düzeltir, ESLint hata verir. Kısır döngüye girersiniz.

Çözüm: ***ESLint'e susmasını söylemek***. ESLint'e diyeceğiz ki: "***Bak kardeş, kodun şekli şemaliyle ilgili (tırnak, virgül, boşluk) ne kadar kuralın varsa hepsini KAPAT. O işi Prettier ustaya bıraktık.***"

Bunun için son bir paket kuruyoruz:

```shell
npm install --save-dev eslint-config-prettier
```

Listing 87 : .eslintignore

Kurulduktan sonra `.eslintrc` dosyanıza gidin ve **extends** kısmının **EN SONUNA** şunu ekleyin

```json
"extends": [
  "plugin:@typescript-eslint/recommended",
  "prettier" // <--- YENİ EKLEDİK!
],
```

Listing 88 : .eslintignore

Artık barış sağlandı. ESLint sadece mantık hatalarına bakacak, Prettier sadece makyajı yapacak.

1.30.13. Klasör Yapısını Düzenleme (`src` ve `dist`)

Arkadaşlar, şu anki klasör yapımız biraz "*Öğrenci Evi*" gibi. Her şey ortada. `app.ts` , `package.json` ile yan yana duruyor. Bunu derlediğimizde (compile ettiğimizde) `.js` dosyaları da buraya saçılacak. Ortalık karışacak.

Hadi bir düzen getirelim:

Kök dizinde `src` (Source - Kaynak) adında bir klasör oluşturun.

Mevcut `app.ts` dosyanızı (varsa `matematik.ts` gibi diğerlerini de) sürükleyip bu `src` klasörünün içine atın.

Artık kuralımız şudur: Bizim yazdığımız her satır kod `src` içinde yaşayacak. Dışarısı sadece ayar dosyalarına (`tsconfig` , `package.json`) ait.

1.30.14. TSConfig Ayarları (`rootDir` ve `outDir`)

Dosyaları taşıdık ama TypeScript'in kafası karıştı. Ona yeni düzeni anlatmamız lazım.

`tsconfig.json` dosyanızı açın. İçinde yüzlerce ayar var ama çoğu yorum satırı (başında // var). Şu iki ayarı bulun, yorum satırlarını kaldırın ve şöyle değiştirin:

```JSON
{
  "compilerOptions": {
    // ...
    "rootDir": "./src",   // "Kodlarım nerede?" -> src içinde
    "outDir": "./dist",   // "Derlenen JS'leri nereye atayım?" -> dist içine
    // ...
  }
}
```

Listing 89 : .eslintignore

Bu ayar hayat kurtarır. Siz kodu derlediğinizde, TypeScript `src` içindeki yapıyı (klasörleri vs.) bozmadan aynen `dist` klasörüne kopyalar ama `.js` 'e çevirerek yapar. Böylece kaynak kodumuzla çalışan kodumuz asla birbirine karışmaz.

1.30.15. Otomasyon: Nodemon Kurulumu

Şimdi size bir soru: Kodda küçücük bir değişiklik yaptığınızda (mesela bir `console.log` eklediğinizde) ne yapıyorsunuz? Terminali durdur **(CTRL+C)**, tekrar `ts-node src/app.ts` yaz...

Günde 100 kere bunu yapmak işkencedir. Bize öyle bir araç lazım ki, biz **CTRL+S (Kaydet)** dediğimiz anda sunucuyu otomatik yeniden başlatsın.

Bu aracın adı: **Nodemon (Node Monitor)**.

Hemen kuralım:

```Shell
npm install --save-dev nodemon ts-node
```

Listing 90 : .eslintignore

(Not: `ts-node` 'u da garanti olsun diye yerel olarak ekledik)

1.30.16. Nodemon Ayarları (nodemon.json)

Nodemon normalde JavaScript dosyalarını izler. Ona "*Sen TypeScript dosyalarına bakacaksın*" dememiz lazım.

Kök dizine `nodemon.json` adında bir dosya açın ve şunu yapıştırın:

```JSON
{
  "watch": ["src"],       // Sadece src klasörünü izle
  "ext": "ts",            // Sadece .ts uzantılı dosyalara bak
  "exec": "ts-node src/app.ts" // Bir değişiklik olursa bu komutu çalıştır
}
```

Listing 91 : .eslintignore

Bu ayar şunu der: *"Gözünü `src` klasöründen ayırma. Orada `.ts` uzantılı bir dosya değişirse, hemen eskiyi öldür ve `ts-node src/app.ts` komutunu yeniden çalıştır."*

Artık her kaydettiğinizde terminalde sunucunun kendine reset attığını göreceksiniz. Hızımıza hız kattık!

1.30.17. NPM Scripts (Otomasyon Komutları)

Arkadaşlar, şu ana kadar terminalde sürekli uzun uzun komutlar yazdık.

Bu yorucu ve bıktırıcı bir işlemdir. Yarın öbür gün bu komutların parametreleri değişirse ne yapacağız? Herkes ezberleyecek mi? Hayır.

Bunun yerine projemizin `package.json` dosyasını açıyoruz. Orada `"scripts"` diye bir bölüm var. Burası projemizin **Kontrol Panelidir**.

Buraya yazdığımız her şeyi `npm run <komut_adi>` diyerek çalıştırabiliriz.

Hadi şu anki `scripts` bölümünü tamamen silin ve benim vereceğim şu standart seti yapıştırın:

```JSON
"scripts": {
  "build": "tsc",
  "format": "prettier --write \"src/**/*.ts\"",
  "start": "ts-node src/app.ts",
  "start:dev": "nodemon",
  "start:prod": "node dist/app.js",
  "lint": "eslint \"{src,apps,libs,test}/**/*.ts\" --fix"
}
```

Listing 92 : .eslintignore

Kaydedin. Şimdi bu anahtarların ve değerlerinin ne işe yaradığını tek tek konuşalım.

1.30.18. Geliştirme Ortamı Komutları (`start:dev`)

En çok kullanacağımız komut: `start:dev` .

Terminali açın ve şunu yazın: `npm run start:dev`

Bunu yazdığınızda arka planda `nodemon` çalışacak. Nodemon da gidip `nodemon.json` ayarlarını okuyacak ve `src` klasörünü izlemeye başlayacak.

Artık projenizi başlatmak için tek bilmeniz gereken komut bu.

Bir de format komutumuz var. Eğer VS Code ayarlarınız bozulursa veya başka bir editör kullanan bir arkadaşınız kodu bozarsa, terminale `npm run format` yazarsınız. Prettier bütün `src` klasörünü tarar ve her şeyi jilet gibi düzeltir.

1.30.19. Canlı Ortam Komutları (`build & start:prod`)

Peki projeyi bitirdik, sunucuya atacağız. Sunucuda `ts-node` kullanır mıyız? **ASLA!** Çünkü `ts-node` yavaştır, her çalıştırdığında derleme yapar.

Sunucuda önce kodu derleriz, sonra saf JavaScript çalıştırırız.

1. `npm run build` : Bu komut tsc'yi çalıştırır. src içindeki TS dosyalarını alır, JS'e çevirip dist klasörüne atar. Deneyin! dist klasörünün dolduğunu göreceksiniz.
2. `npm run start:prod` : Bu komut ise node dist/app.js çalıştırır. Yani derlenmiş, optimize edilmiş, saf JavaScript kodunu ayağa kaldırır.

Prodüksiyon (Canlı) ortamında sadece bu ikisini kullanacağız.

1.30.20. Linting (Sağlık Kontrolü)

Son komutumuz: `npm run lint` .

Bu komut, projedeki bütün dosyaları ESLint polisine taratır. Eğer `--fix` parametresini eklediyseniz (ki ekledik), ESLint bulduğu basit hataları (noktalı virgül eksikliği, yanlış boşluklar vb.) kendi kendine düzeltir. Düzeltemediklerini (mantık hatalarını) ise size raporlar.

Genelde projeyi Git'e (Github/Gitlab) göndermeden önce mutlaka bir kere `npm run lint` çalıştırırız ki, bozuk kod göndermeyelim.

Evet arkadaşlar, artık tam teçhizatlı bir geliştirme ortamımız var.

1. Kodumuz otomatik formatlanıyor (Prettier).
2. Hatalarımız denetleniyor (ESLint).
3. Sunucumuz otomatik yeniden başlıyor (Nodemon).
4. Tek komutla derleyip çalıştırabiliyoruz (NPM Scripts).

NestJS'e başlamak için her şey hazır.

1.30.21. Git Başlatma (Versiyon Kontrolü)

Geliştirme ortamımız hazır. Şimdi bu projeyi kayıt altına almamız gerekiyor. Yazılım geliştirmede yazdığınız kodları bilgisayarınızdaki bir klasörde tutmak yeterli değildir; bir versiyon kontrol sistemi kullanmalısınız.

Terminali açın ve proje klasörünün içindeyken şu komutu yazarak Git'i başlatın:

```shell
git init
```

Listing 93 : .eslintignore

Bu komut, projenizin bulunduğu klasörde gizli bir `.git` klasörü oluşturur ve artık bu dizindeki değişiklikleri takip etmeye başlar. VS Code'un sol menüsünde, 'Source Control'

sekmesinde binlerce dosyanın listelendiğini göreceksiniz. Bu normaldir, çünkü henüz hangi dosyaların takip edileceğini, hangilerinin edilmeyeceğini belirtmedik.

1.30.22. .gitignore Dosyası Oluşturma

Bir TypeScript projesinde asla Git'e yüklenmemesi gereken iki temel klasör vardır:

1. `node_modules` : Bu klasör yüzlerce megabayt boyutundadır ve internetten indirilebilir kütüphaneleri içerir. Bunu depoya yüklemek hem yükleme süresini uzatır hem de gereksizdir. Başka bir geliştirici projeyi indirdiğinde npm install diyerek bunları zaten oluşturabilir.
2. `dist` : Bu klasör, TypeScript kodlarının derlenmiş halidir. Kaynak kod (src) bizde olduğu sürece, `dist` klasörü her zaman yeniden üretilebilir. Bu yüzden takip edilmesine gerek yoktur.

Bu dosyaları Git'in görmezden gelmesi için kök dizinde `.gitignore` adında bir dosya oluşturun ve içine şunları yazın:

```text
node_modules
dist
.env
```

Listing 94 : .eslintignore

(Not: `.env` dosyasını da güvenlik nedeniyle şimdiden ekliyoruz, ileride kullanacağız.)

1.30.23. İlk Kayıt (Commit)

`.gitignore` dosyasını kaydettiğinizde, VS Code'daki değişen dosya sayısının (Source Control sekmesinde) binlerden, sadece bizim oluşturduğumuz 5-10 dosyaya düştüğünü göreceksiniz. Doğrusu budur.

Şimdi bu başlangıç noktasını kaydedelim. Terminale sırasıyla şu komutları yazın:

```shell
git add .
git commit -m "Proje kurulumu: TypeScript, ESLint, Prettier ve Nodemon
yapılandırması tamamlandı"
```

Listing 95 : .eslintignore

Böylece projemizin ilk güvenli yedeğini almış olduk. Kodda bir hata yaparsanız bu noktaya geri dönebilirsiniz.

1.31. 1. Hafta Proje Ödevi Tanıtımı

Konu anlatımını tamamladık. Kalan sürede, önümüzdeki haftaya kadar yapmanızı istediğim **haftalık projeyi** açıklayacağım. Bu ödev, kurduğumuz geliştirme ortamını ve TypeScript temellerini pekiştirmeniz için zorunludur.

Projemizin adı: **Basit Görev Yöneticisi (Task Manager CLI)**.

Sizden konsol üzerinde çalışan, basit bir yapılacaklar listesi uygulaması istiyorum. Herhangi bir veritabanı kullanmayacağız, verileri şimdilik bellekte (bir dizi içinde) tutacağız.

1.31.0.1. Teknik Gereksinimler

Projeyi oluştururken şu teknik kurallara uymanız gerekiyor:

1. **Klasör Yapısı:** Az önce kurduğumuz `src` , `dist` yapısını ve `tsconfig.json` ayarlarını birebir uygulayın.
2. **Interface Kullanımı:** Bir Gorev (Task) interface'i tanımlayın. İçinde şu alanlar olmalı:
   - `id` (number)
   - `baslik` (string)
   - `durum` (Enum: 'Yapilacak', 'Sürüyor', 'Tamamlandi')
   - `olusturulmaTarihi` (Date)
3. **Class Yapısı:** `GorevYoneticisi` adında bir sınıf yazın. Bu sınıf içinde görev ekleme, silme ve listeleme metodları bulunmalı.

1.31.0.2. Kod Kalitesi ve Kısıtlamalar

Kodunuzu yazarken şu kısıtlamalara dikkat edin:

- `any` **Tipi Yasak:** Hiçbir yerde `any` tipi kullanmayacaksınız. ESLint ayarlarınızda `no-explicit-any` kuralı açık olmalı.
- **Method Dönüş Tipleri:** Yazdığınız her fonksiyonun dönüş tipi (Return Type) mutlaka belirtilmeli (Örn: `void` , : `Gorev[]`).
- **Private/Public:** Sınıf içindeki görev listesi dizisi (tasks array) mutlaka `private` olmalı, dışarıdan doğrudan erişilememeli.

1.31.0.3. Teslim Formatı

Projeyi tamamladığınızda şu adımları takip ederek teslim edeceksiniz:

1. Projenizi `npm run build` komutuyla derleyin. Hata almadığınızdan emin olun.
2. `npm run lint` komutunu çalıştırın. Konsolda hiç hata veya uyarı görünmemeli.
3. Projenizi bir GitHub deposuna (repository) yükleyin.
4. Repo linkini bir sonraki derse gelmeden hazır edin.

Kalan sürede sınıf içinde örnek bir Gorev interface'i tasarlayarak başlayabiliriz.

1.31.0.4. Enum ve Interface Tanımlama

Ödevde istediğim `Gorev` yapısını birlikte kuralım. Öncelikle görevlerin durumunu belirtecek olan sabitleri, yani `Enum` yapısını oluşturuyoruz.

`src` klasörü içine `types.ts` adında yeni bir dosya açın. Tipleri ayrı dosyada tutmak düzen açısından önemlidir.

```typescript
export enum GorevDurumu {
    YAPILACAK = 'Yapılacak',
    SURUYOR = 'Sürüyor',
    TAMAMLANDI = 'Tamamlandı'
}

export interface Gorev {
    id: number;
    baslik: string;
    durum: GorevDurumu; // Enum tipini burada kullanıyoruz
    tarih: Date;
}
```

Listing 96 : .eslintignore

Burada `status` alanının tipi string değil, `GorevDurumu` oldu. Bu sayede kodun ilerleyen kısımlarında `'Yapılıyor'` mu yazdım, `'Sürüyor'` mu yazdım karmaşası yaşamayacağız.